SMARTPARCEL
AWS SERVERLESS PROJECT REPORT

NET 214 - Network Programming | Spring 2026

Student Name	Khalid Al Marri
Student ID	20200001899
CUD Email	20200001899@students.cud.ac.ae
AWS Resource Suffix	khalid20200001899
AWS Region	Asia Pacific (Sydney) - ap-southeast-2
AWS Account ID	598892455787
GitHub Repository	https://github.com/KhalidAlMarri96/smartparcel-net214-20200001899-final

Serverless Architecture - API Gateway - Lambda - DynamoDB - S3 - SQS - SNS

Table of Contents

1. Introduction
1.1 High-Level Architecture
2. Phase 1 - Serverless REST API
2.1 DynamoDB Table
2.2 S3 Bucket
2.3 Lambda API Function
2.4 API Gateway
2.5 API Key and Role-Based Access
3. Phase 2 - Event-Driven Notification Pipeline
3.1 SQS Queue and Dead Letter Queue
3.2 Lambda Notifier
3.3 SNS Notification
3.4 DynamoDB Global Secondary Index
3.5 Concurrent Load Test
4. Phase 3 - Secure Network Architecture
4.1 VPC, Subnets, Internet Gateway, and Route Table
4.2 Network ACL
4.3 VPC Gateway Endpoint for DynamoDB
4.4 IAM Least Privilege
4.5 Input Validation and Error Handling
4.6 CloudWatch Monitoring
4.7 CloudTrail Auditing
5. Phase 4 - Architecture and Well-Architected Review
5.1 CloudFormation Template
5.2 Network Design
5.3 Security Design
5.4 High Availability and Disaster Recovery
5.5 Database Choice
5.6 Cost Analysis
5.7 Well-Architected Framework Review
6. Conclusion
Appendices - Source Code

1. Introduction

SmartParcel is a serverless parcel delivery system deployed on AWS. The system allows delivery drivers to create parcels and update delivery status, customers to check parcel status, and administrators to list and manage parcels. The application uses managed AWS services so the system can scale without managing EC2 servers.

The project was deployed in Asia Pacific (Sydney), ap-southeast-2. All main AWS resource names include the student identifier suffix khalid20200001899.

Item	Details
Main stack	Khalid-FINAL
Region	ap-southeast-2
Resource suffix	khalid20200001899
Main API URL	https://grjyln0fw8.execute-api.ap-southeast-2.amazonaws.com/prod
Main tested parcel	PKG-2026-25581CD7

1.1 High-Level Architecture

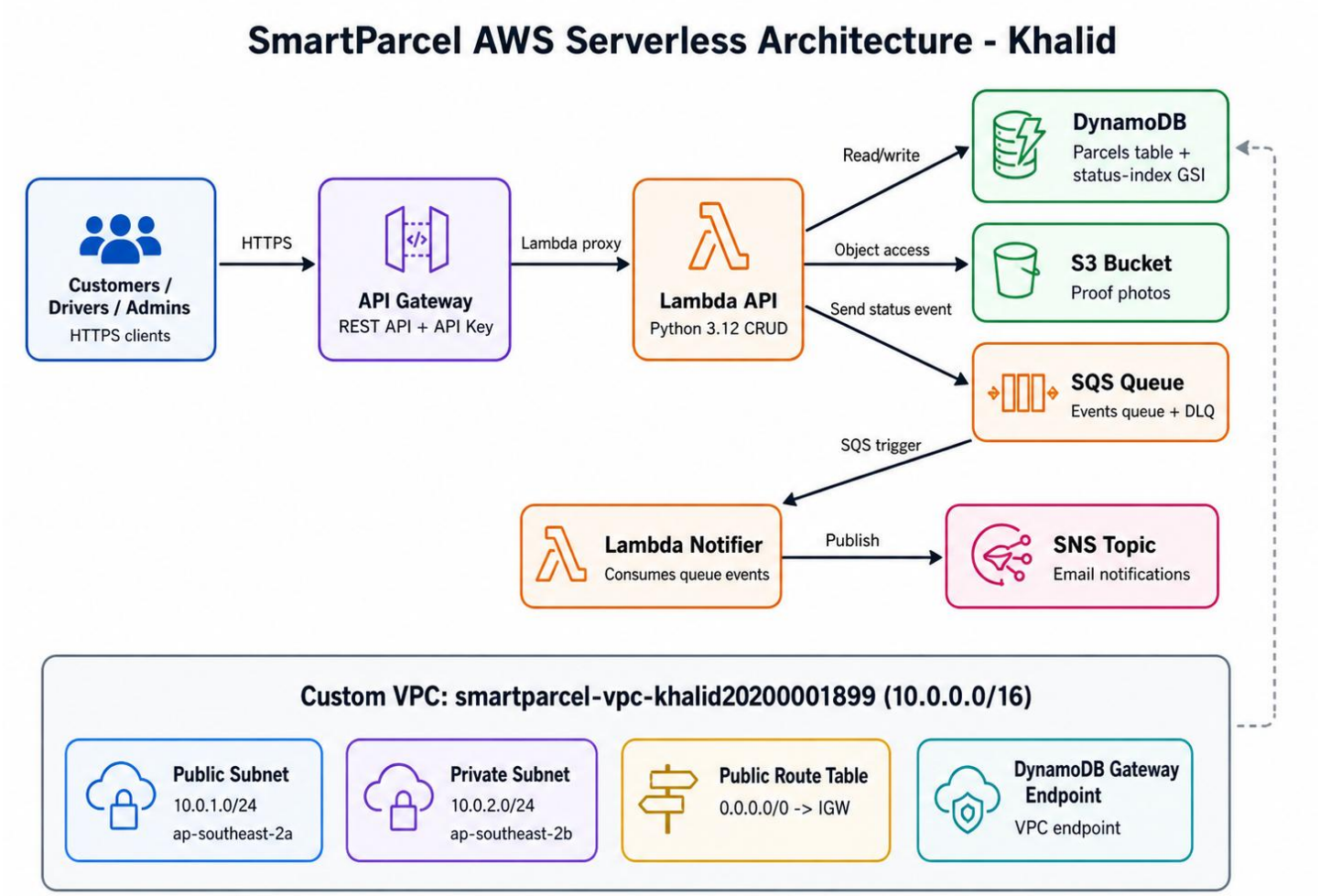


Figure 1: SmartParcel high-level serverless architecture

The user sends HTTPS requests to API Gateway. API Gateway invokes the API Lambda. The API Lambda validates input and role, stores parcel records in DynamoDB, and sends status-change events to SQS. SQS triggers the notifier Lambda, which publishes the message to SNS for email notification. S3 stores delivery proof photos securely.

2. Phase 1 - Serverless REST API

2.1 DynamoDB Table

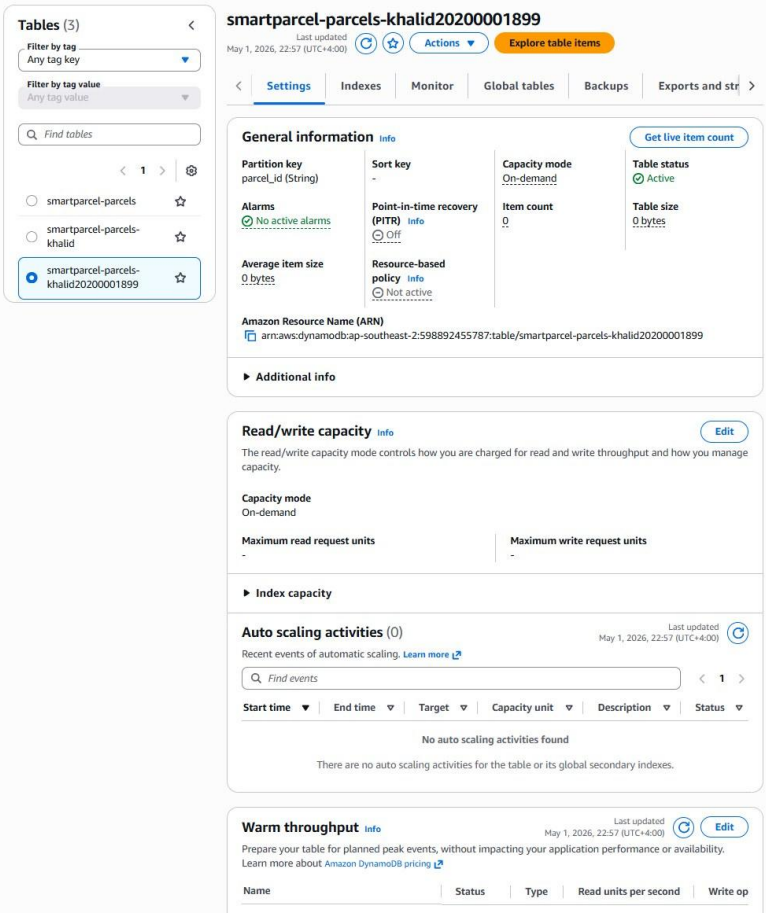


Figure 2: DynamoDB table overview

What I did: Created smartparcel-parcels-khalid20200001899 with partition key parcel_id and on-demand capacity. Why I did it: Parcel records need fast key-value lookup by parcel ID. What this AWS service does: DynamoDB is a managed NoSQL database. How it connects: The API Lambda creates, reads, updates, deletes, and queries parcel records.

2.2 S3 Bucket

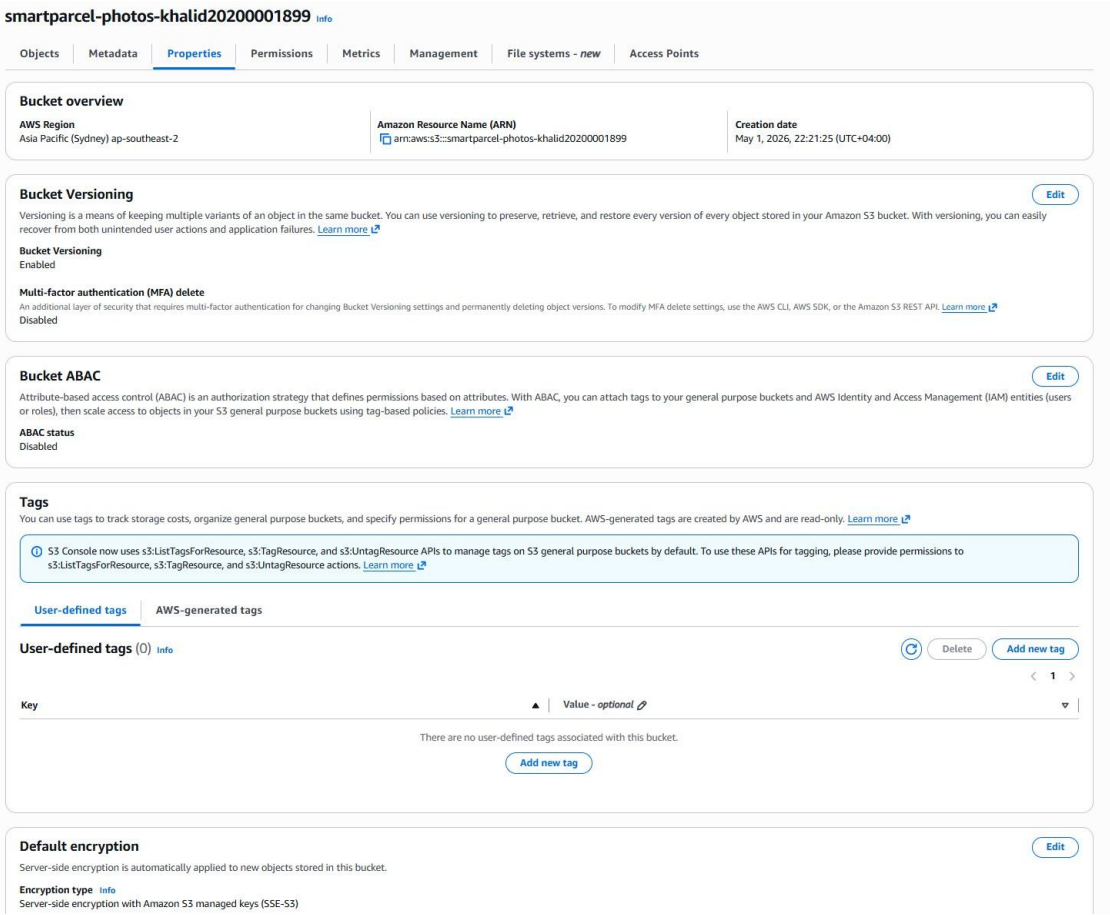


Figure 3: S3 bucket versioning and SSE-S3 encryption

What I did: Created smartparcel-photos-khalid20200001899 with versioning enabled and SSE-S3 encryption. Why I did it: Proof photos should be protected and recoverable. What S3 does: S3 stores objects such as delivery proof images. How it connects: Lambda can store proof photos in S3 while DynamoDB stores metadata.

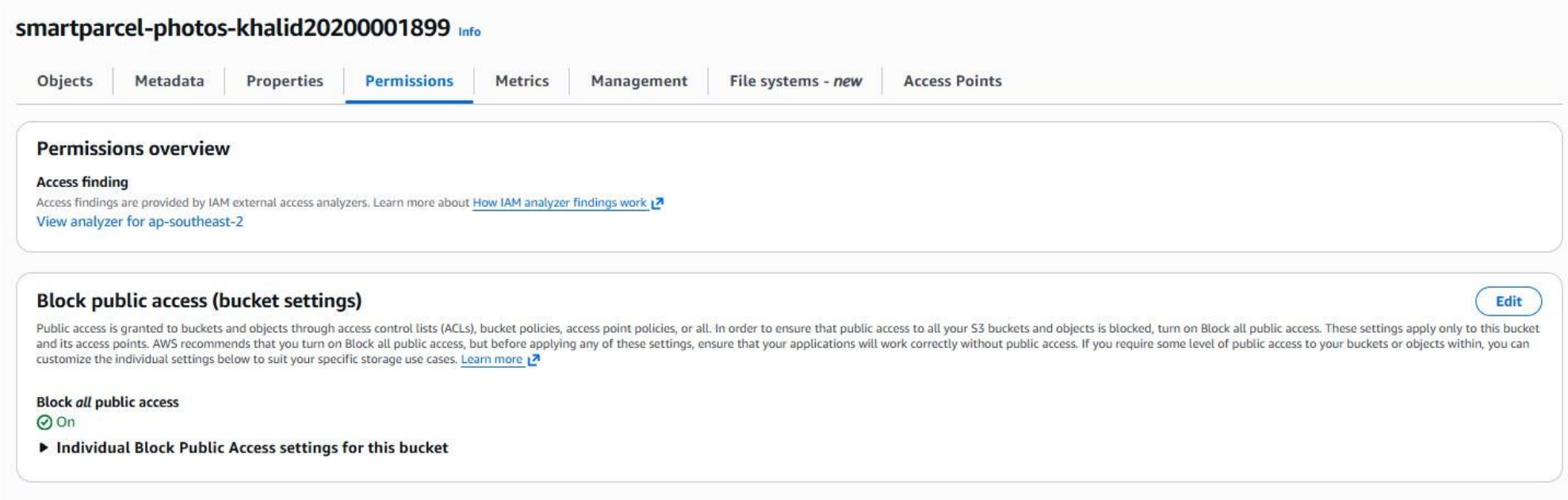


Figure 4: S3 block public access enabled

Block all public access is enabled so proof photos are not exposed publicly.

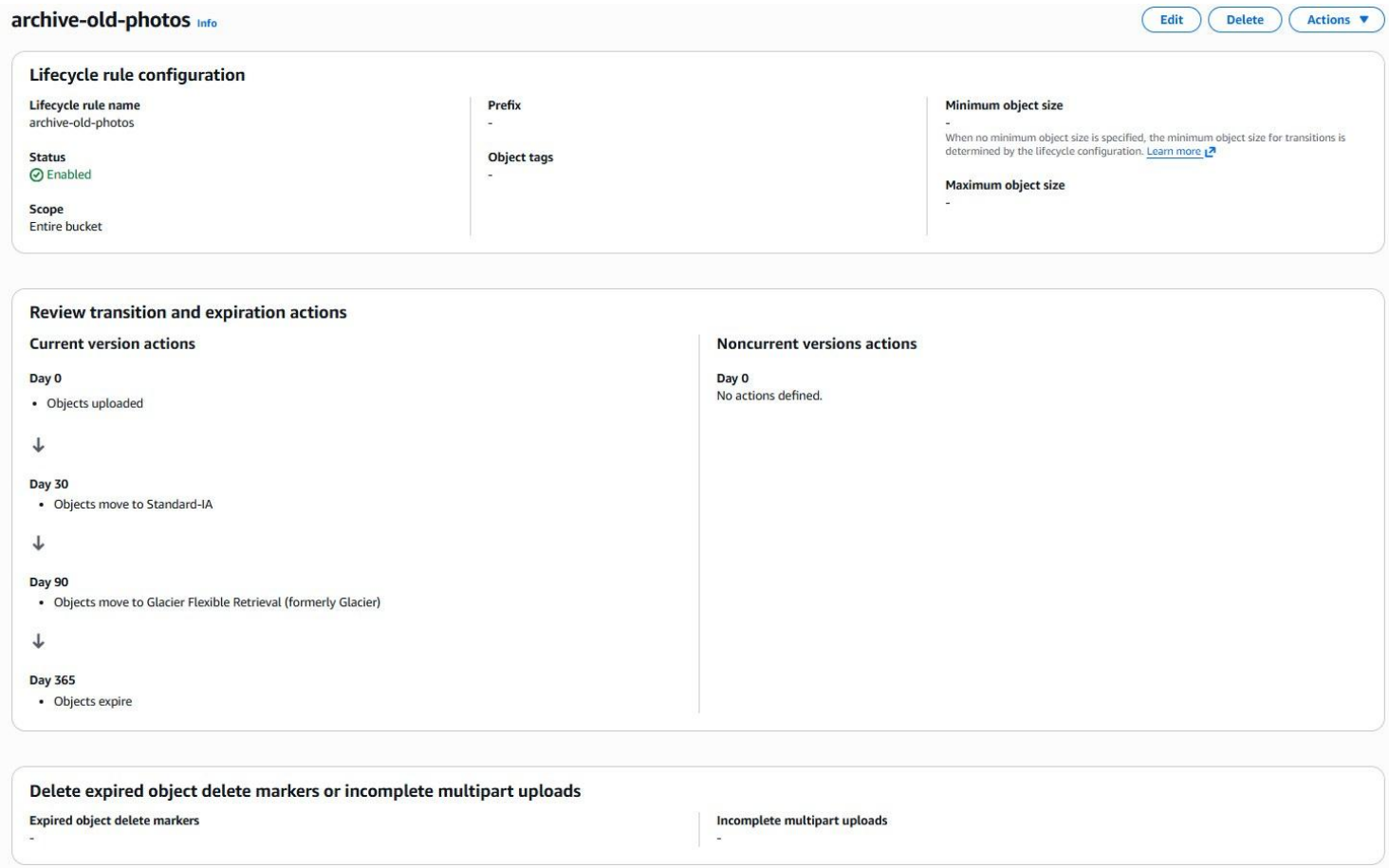


Figure 5: S3 lifecycle rule archive-old-photos

The lifecycle rule moves photos to Standard-IA after 30 days, Glacier after 90 days, and expires them after 365 days to reduce cost.

2.3 Lambda API Function

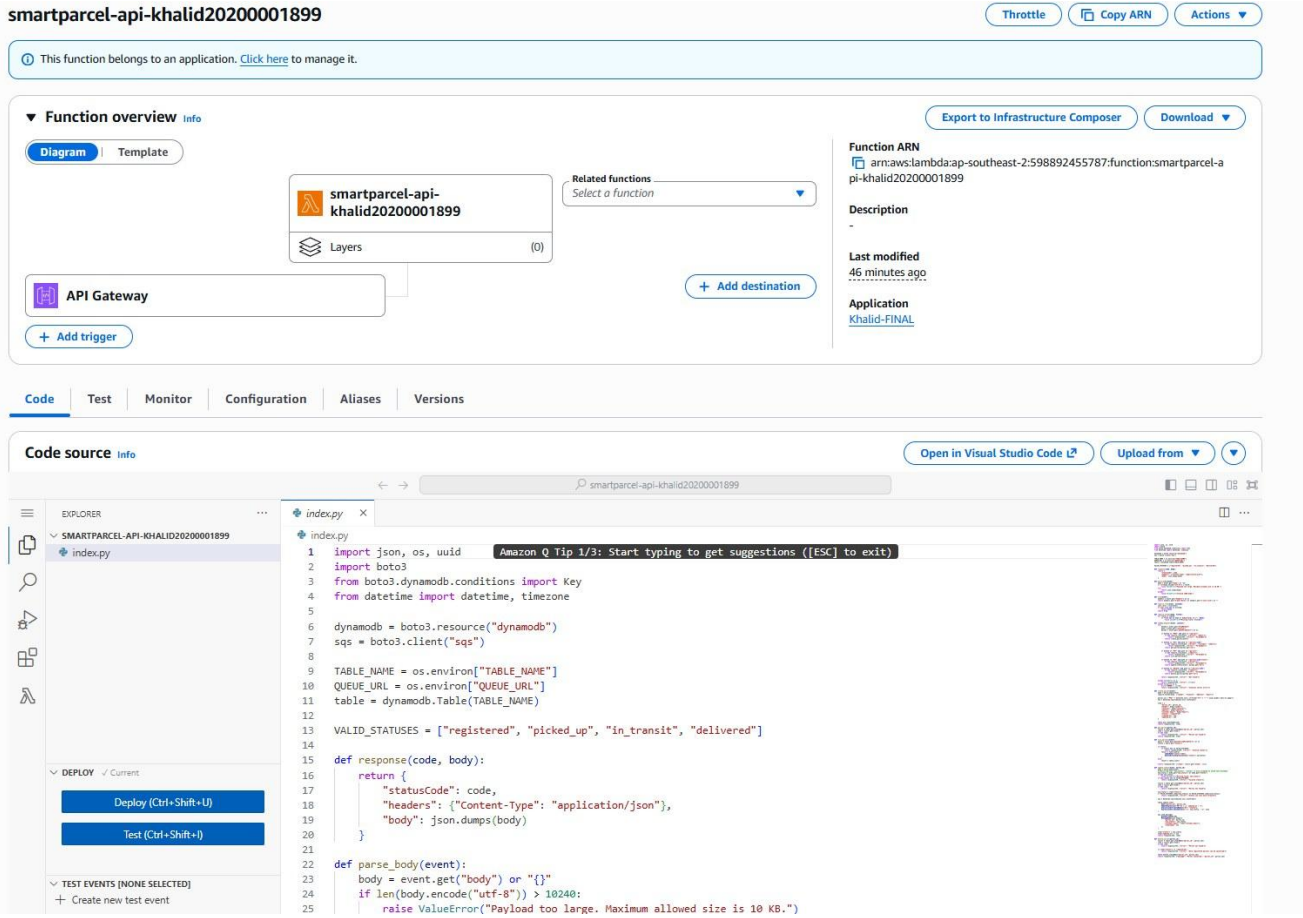


Figure 6: Lambda API function overview and code

The API Lambda uses Python 3.12 and is triggered by API Gateway. It handles the CRUD operations for SmartParcel and communicates with DynamoDB, SQS, and S3.

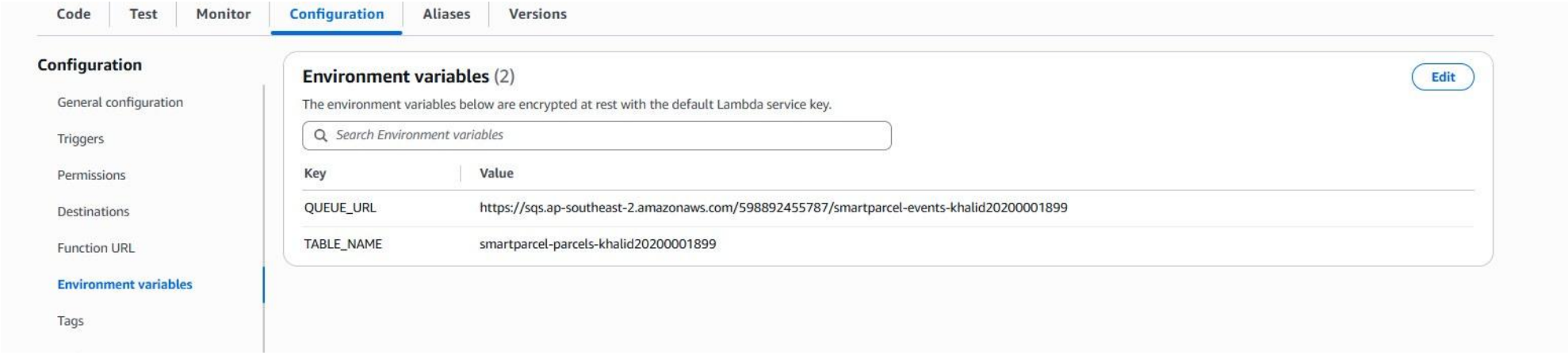


Figure 7: Lambda API environment variables

TABLE_NAME connects the function to DynamoDB and QUEUE_URL connects it to SQS.

2.4 API Gateway

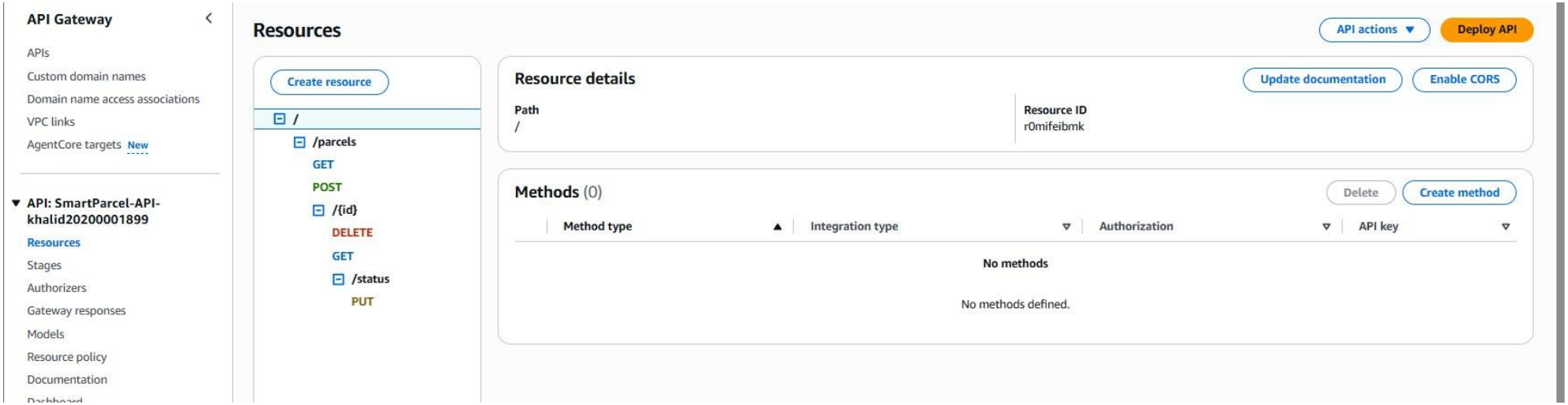


Figure 8: API Gateway resources and methods

API Gateway exposes REST endpoints: POST /parcels, GET /parcels, GET /parcels/{id}, PUT /parcels/{id}/status, and DELETE /parcels/{id}.

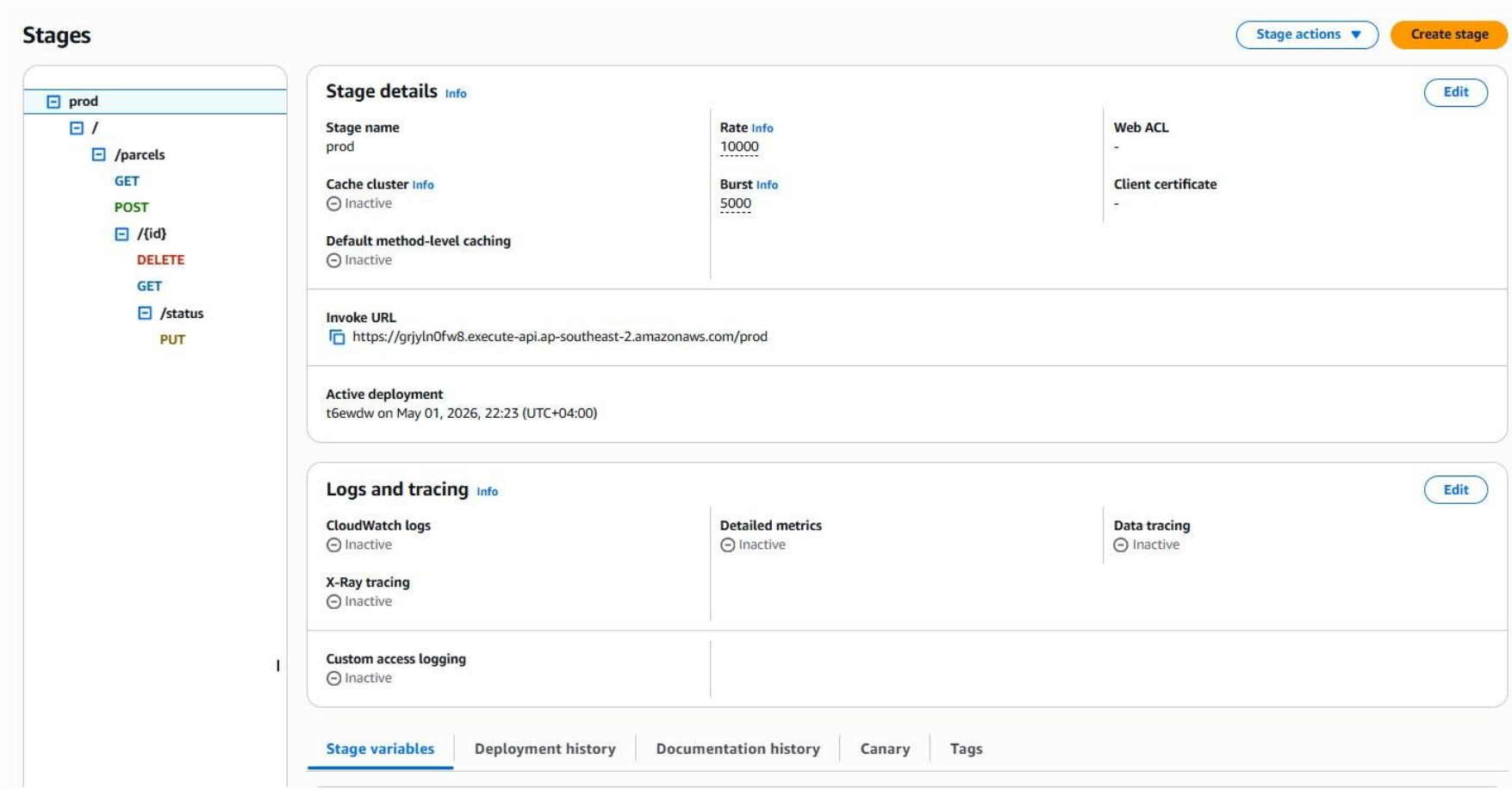


Figure 9: API Gateway prod stage and invoke URL

The REST API is deployed to the prod stage. The invoke URL is used for curl testing and live demo.

```
>   -d "{\"sender\":\"Khalid\",\"receiver\":\"Sara\",\"address\":\"Dubai Mall\",\"email\":\"$EMAIL\"}" | tee create.txt
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   376 100   275 100   101   278   102  --:--:-- --:--:-- --:--:--   381
HTTP/2 201
content-type: application/json
content-length: 275
date: Fri, 01 May 2026 18:44:25 GMT
x-amzn-trace-id: Root=1-69f4f488-6d99ccf5232589a61843cc0e;Parent=5784af93811abc65;Sampled=0;Lineage=1:339fb8a1:0
x-amzn-requestid: 752891b6-ec7a-406b-b046-b979d721bd81
x-amz-apigw-id: csslUFoVywMEdXA=
x-cache: Miss from cloudfront
via: 1.1 eb3589b1230a45883fc0813bdb92ed5e.cloudfront.net (CloudFront)
x-amz-cf-pop: SYD62-P1
x-amz-cf-id: jdWVjwtOma7vyHMjrBz8PRcXiqfRoG13yHEolhwzSb2UNb55soiGUg==

{"parcel_id": "PKG-2026-25581CD7", "sender": "Khalid", "receiver": "Sara", "address": "Dubai Mall", "customer_email": "20200001899@students.cud.ac.ae", "status": "registered", "created_at": "2026-05-01T18
:44:24.738612+00:00", "updated_at": "2026-05-01T18:44:24.738612+00:00"}~ $
```

Figure 10: POST /parcels creates parcel

POST /parcels returned HTTP 201 and created parcel PKG-2026-25581CD7 with status registered.

```
~ $ curl -i "$API/parcels/$PID" \
>   -H "x-api-key: $KEY" \
>   -H "X-User-Role: customer" | tee get.txt
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   275 100   275   0     0   975     0  --:--:-- --:--:-- --:--:--   978
HTTP/2 200
content-type: application/json
content-length: 275
date: Fri, 01 May 2026 18:47:14 GMT
x-amzn-trace-id: Root=1-69f4f532-1ff53bc71bab710c20c7423b;Parent=7502661cf6565b0a;Sampled=0;Lineage=1:339fb8a1:0
x-amzn-requestid: e10d51d4-2fc2-4b32-9949-fff1f390089c
x-amz-apigw-id: css_-EHFywMEE0g=
x-cache: Miss from cloudfront
via: 1.1 1d3e75fe2262e8a6f4a318b2bf3e6570.cloudfront.net (CloudFront)
x-amz-cf-pop: SYD62-P1
x-amz-cf-id: mH_1j4yuHePA7LNYyiP--96a8TskSJemWhvr5Nr_CY6BzVFRcoQU7w==

{"receiver": "Sara", "updated_at": "2026-05-01T18:44:24.738612+00:00", "sender": "Khalid", "created_at": "2026-05-01T18:44:24.738612+00:00", "parcel_id": "PKG-2026-25581CD7", "status": "registered", "addr
ess": "Dubai Mall", "customer_email": "20200001899@students.cud.ac.ae"}~ $
```

Figure 11: GET /parcels/{id} returns parcel

GET by parcel ID returned HTTP 200 and the same parcel details.

```
>   -H "X-User-Role: admin" | python3 -m json.tool | tee list_pretty.json
{
    "items": [
        {
            "receiver": "Sara",
            "sender": "Khalid",
            "updated_at": "2026-05-01T18:48:11.174538+00:00",
            "created_at": "2026-05-01T18:44:24.738612+00:00",
            "parcel_id": "PKG-2026-25581CD7",
            "status": "picked_up",
            "address": "Dubai Mall",
            "customer_email": "20200001899@students.cud.ac.ae"
        }
    ]
}
~ $
```

Figure 12: GET /parcels admin list pretty JSON

The admin list endpoint returns a JSON items array with the parcel record.

```
x-amz-cf-id: 2xJumaKbQXiFMD-w6dcdqtaTLjpcc22nv_GeEXrQBxySyjHwrhIXng==

{"error": "Forbidden"}~ $
~ $
~ $ curl -i -X DELETE "$API/parcels/$PID" \
>   -H "x-api-key: $KEY" \
>   -H "X-User-Role: admin"
HTTP/2 409
content-type: application/json
content-length: 53
date: Fri, 01 May 2026 18:52:38 GMT
x-amzn-trace-id: Root=1-69f4f676-04f03f1527f3b93a2a1ed9f9;Parent=46974e136031d0f0;Sampled=0;Lineage=1:339fb8a1:0
x-amzn-requestid: 38300c58-6b77-4588-ae19-5baf0ba60e10
x-amz-apigw-id: cstykFKRywMEERA=
x-cache: Error from cloudfront
via: 1.1 bac8af6ab43417aff0768ef23a8c05de.cloudfront.net (CloudFront)
x-amz-cf-pop: SYD62-P1
x-amz-cf-id: u6Qv1CRvtw3LECqrsKQncrqqObp3fVeFfAXhgo6Hb9ZEwqBqt2Z_gA==

{"error": "Only registered parcels can be cancelled"}~ $
```

Figure 13: DELETE picked_up parcel returns 409

DELETE returned 409 because the parcel was already picked_up. This proves only registered parcels can be cancelled.

2.5 API Key and Role-Based Access

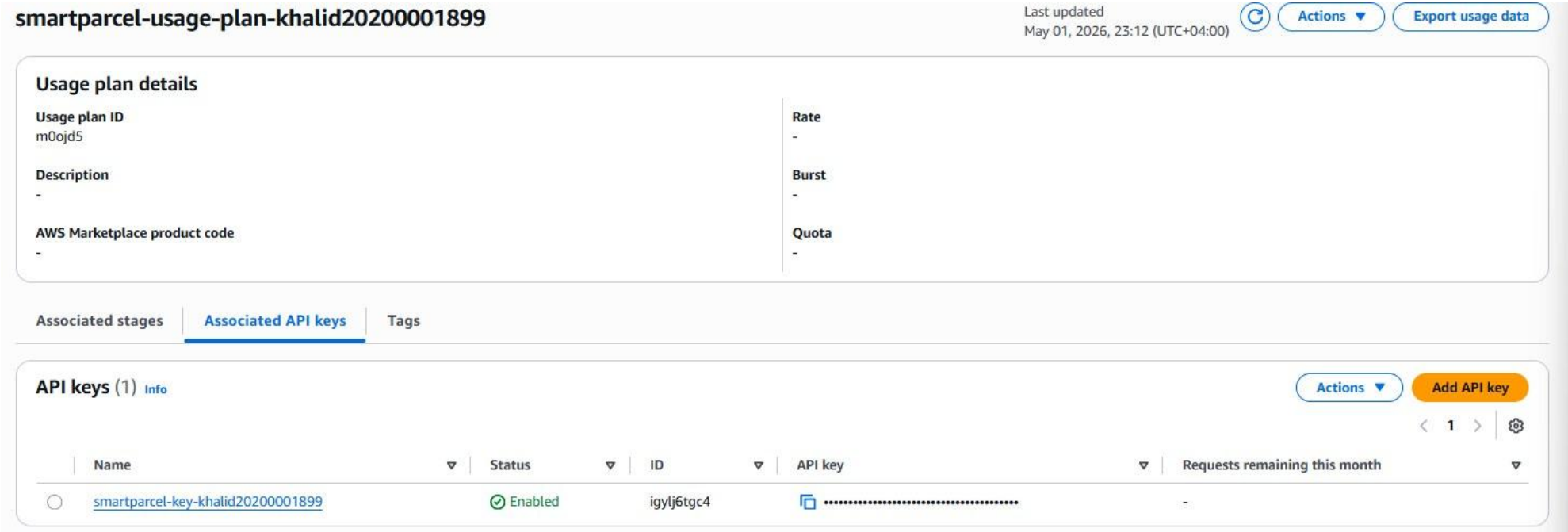


Figure 14: Usage plan associated API key

The API key is enabled and connected to the usage plan.

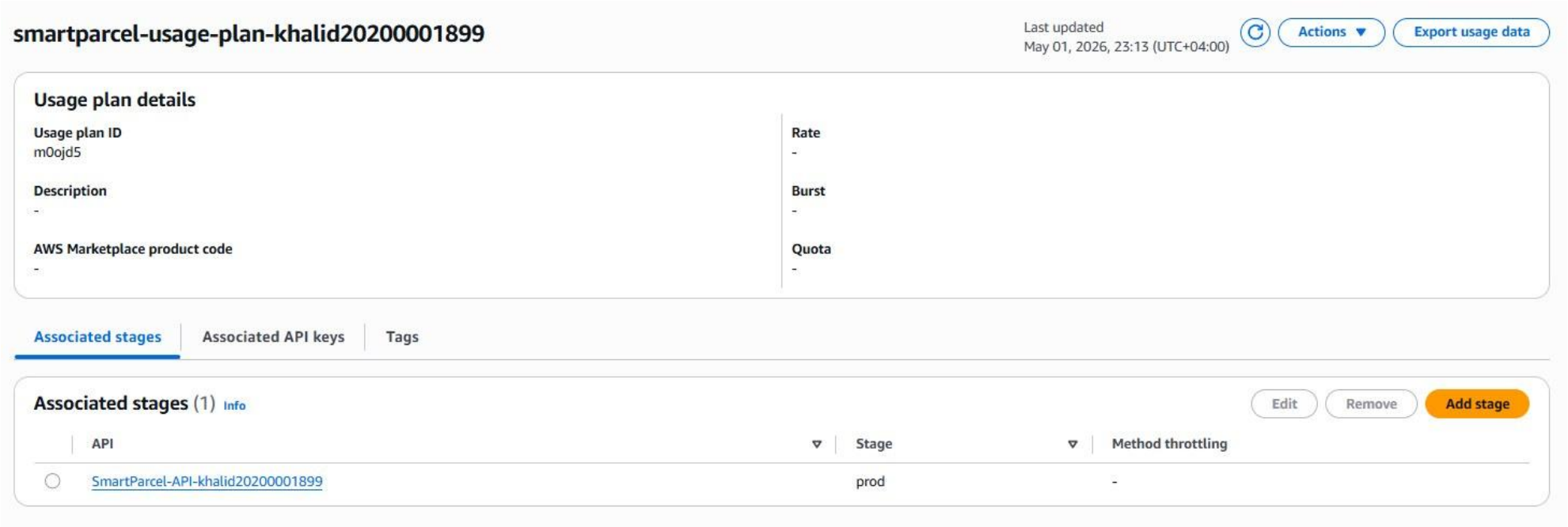


Figure 15: Usage plan associated prod stage

The usage plan is attached to the SmartParcel REST API prod stage.

```
~ $ smartparcel20200001899_khalid.yaml
-bash: smartparcel20200001899_khalid.yaml: command not found
~ $
~ $
~ $ API="https://grjyln0fw8.execute-api.ap-southeast-2.amazonaws.com/prod"
~ $ KEY="khalid-20200001899-smartparcel-key-2026"
~ $ curl -i -X POST "$API/parcels" \
>   -H "X-User-Role: driver" \
>   -H "Content-Type: application/json" \
>   -d '{"sender":"Khalid","receiver":"Test Receiver","address":"Dubai","email":"20200001899@studnts.cud.ac.ae"}'
HTTP/2 403
content-type: application/json
content-length: 23
date: Fri, 01 May 2026 18:26:08 GMT
x-amz-apigw-id: csp6IFNlSwMEEww=
x-amzn-requestid: 8757a9e5-997e-4436-8651-3037653b832a
x-amzn-errortype: ForbiddenException
x-cache: Error from cloudfront
via: 1.1 8bec138951dfffa4e8e0ac983bb30e76.cloudfront.net (CloudFront)
x-amz-cf-pop: SYD62-P1
x-amz-cf-id: _cfpJR_d6eW_-pxQna7Tovm4KadUEPTYCi87A7-qpsk-KZ7qScqgTA==

{"message":"Forbidden"}~ $
```

Figure 16: Request without API key returns 403 Forbidden

A request without x-api-key was blocked with 403 Forbidden.

```
~ $
~ $ curl -i "$API/parcels" \
>   -H "x-api-key: $KEY" \
>   -H "X-User-Role: customer"
HTTP/2 403
content-type: application/json
content-length: 22
date: Fri, 01 May 2026 18:51:26 GMT
x-amzn-trace-id: Root=1-69f4f62e-548b139434cd96cb1b76399a;Parent=178b3ecc736bf0bb;Sampled=0;Lineage=1:339fb8a1:0
x-amzn-requestid: c52683ff-0382-4181-a644-7a076f80041e
x-amz-apigw-id: cstnVGTHSwMEAVw=
x-cache: Error from cloudfront
via: 1.1 e3f64b5e1795622ac1fd367fad798c10.cloudfront.net (CloudFront)
x-amz-cf-pop: SYD62-P1
x-amz-cf-id: 2xJumaKbQXiFMD-w6dcdqtaTLjpcc22nv_GeEXrQBxySyjHwrhIXng==

{"error": "Forbidden"}~ $
```

Figure 17: Customer role cannot list all parcels

Lambda role logic blocks customer role from admin-only GET /parcels and returns 403 Forbidden.

3. Phase 2 - Event-Driven Notification Pipeline

3.1 SQS Queue and Dead Letter Queue

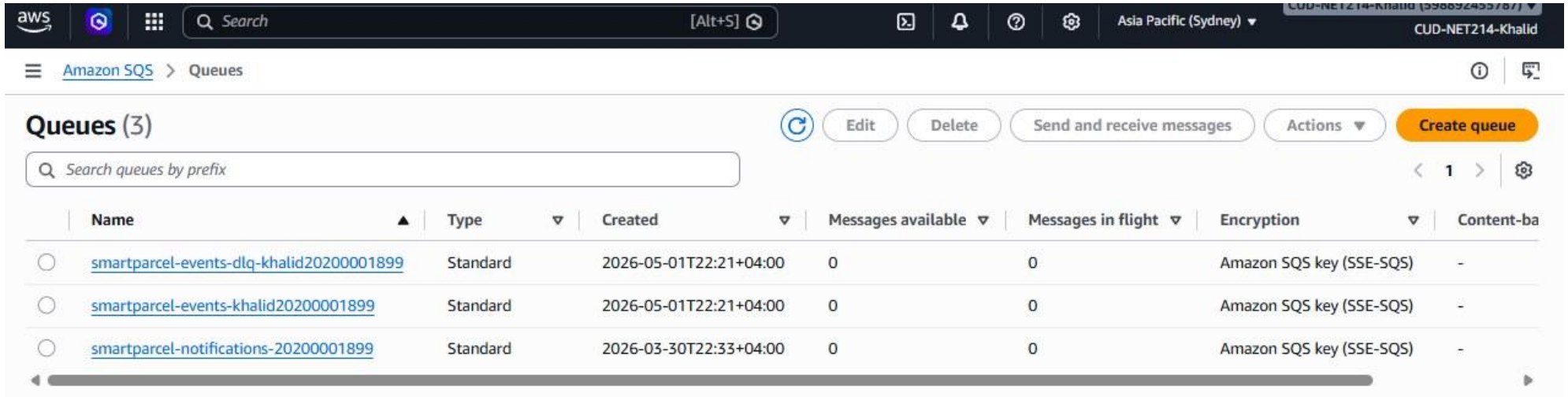


Figure 18: SQS main queue and DLQ

The main events queue and DLQ were created. SQS decouples API requests from notification processing.

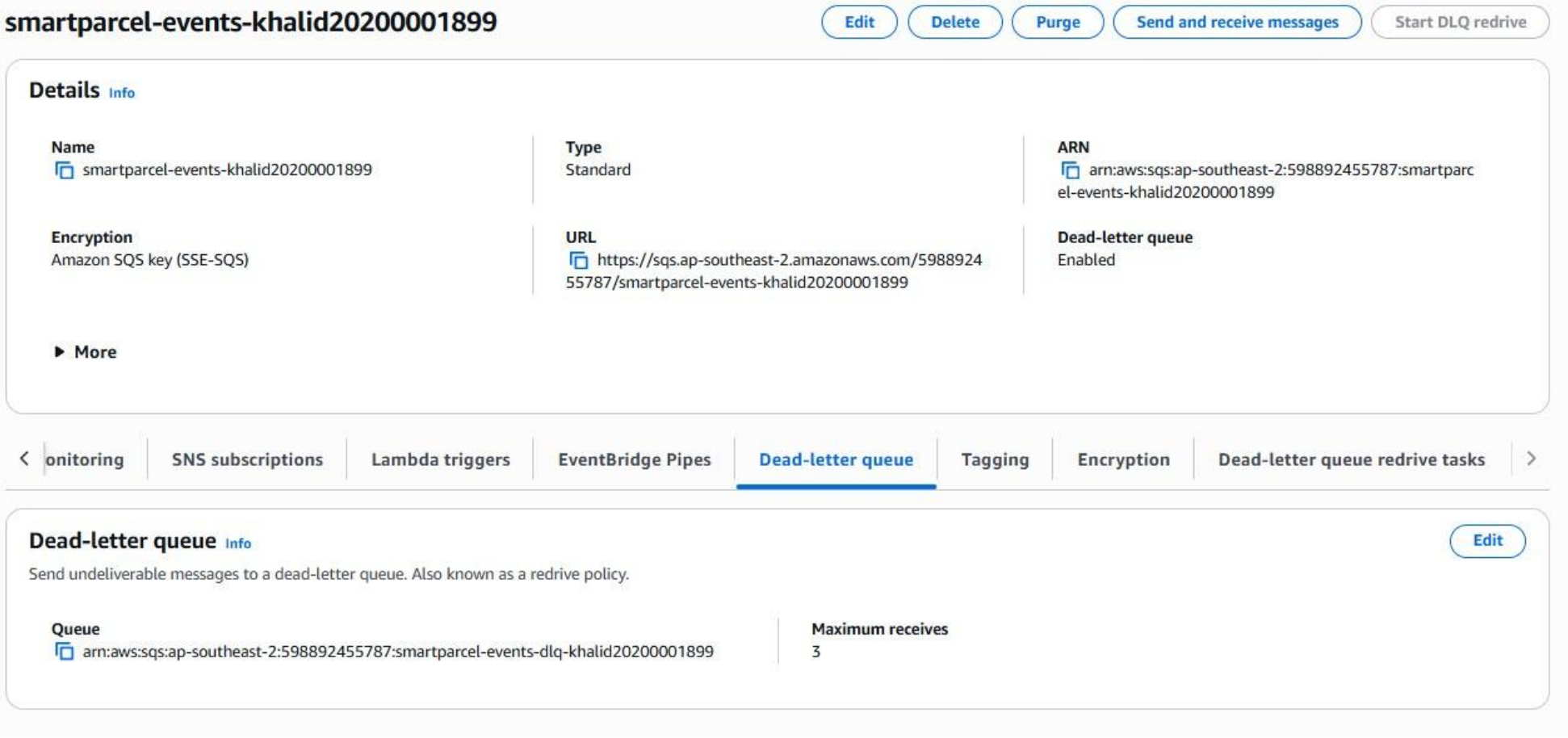


Figure 19: SQS redrive policy maxReceiveCount 3

Messages that fail three times move to the DLQ, preventing endless retries.

```
>   -H "X-API-Key: $KEY" \
>   -H "X-User-Role: driver" \
>   -H "Content-Type: application/json" \
>   -d '{"new_status":"picked_up"}' | tee update.txt
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   300 100   274 100    26   771    73  --:--:-- --:--:-- --:--:--   845
HTTP/2 200
content-type: application/json
content-length: 274
date: Fri, 01 May 2026 18:48:11 GMT
x-amzn-trace-id: Root=1-69f4f56b-44db640a6f3e604271b2a3e0;Parent=73e9513dbe542ae8;Sampled=0;Lineage=1:339fb8a1:0
x-amzn-requestid: d30a629a-6347-44b1-97cb-2f948c64515f
x-amz-apigw-id: cstIyGhwSwMEmTA=
x-cache: Miss from cloudfront
via: 1.1 0c5c9092233f69156c68308fd823bd58.cloudfront.net (CloudFront)
x-amz-cf-pop: SYD62-P1
x-amz-cf-id: 54J-PyO_m74rEKvHRN4voVNxvwqTmGSTGQZM0oLHcNtckZpuUwiC4g==

{"receiver": "Sara", "updated_at": "2026-05-01T18:48:11.174538+00:00", "sender": "Khalid", "created_at": "2026-05-01T18:44:24.738612+00:00", "parcel_id": "PKG-2026-25581CD7", "status": "picked_up", "addre
ss": "Dubai Mall", "customer_email": "20200001899@students.cud.ac.ae"}~ $
```

Figure 20: PUT status update picked_up

PUT /parcels/{id}/status changed the status to picked_up and triggered the notification pipeline.

3.2 Lambda Notifier

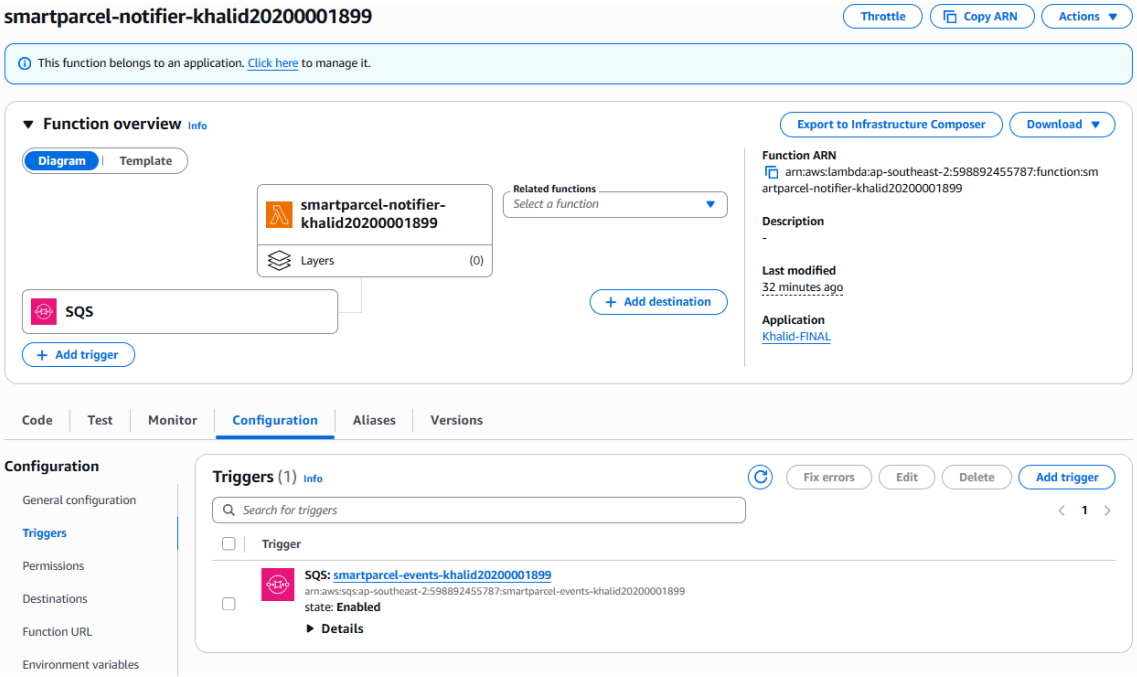


Figure 21: Lambda Notifier with SQS trigger enabled

The notifier Lambda is connected to the SQS events queue through an enabled trigger. This proves that when the API Lambda sends a status-change message to SQS, AWS automatically invokes the notifier Lambda to process the event and publish the email notification through SNS.

3.3 SNS Notification

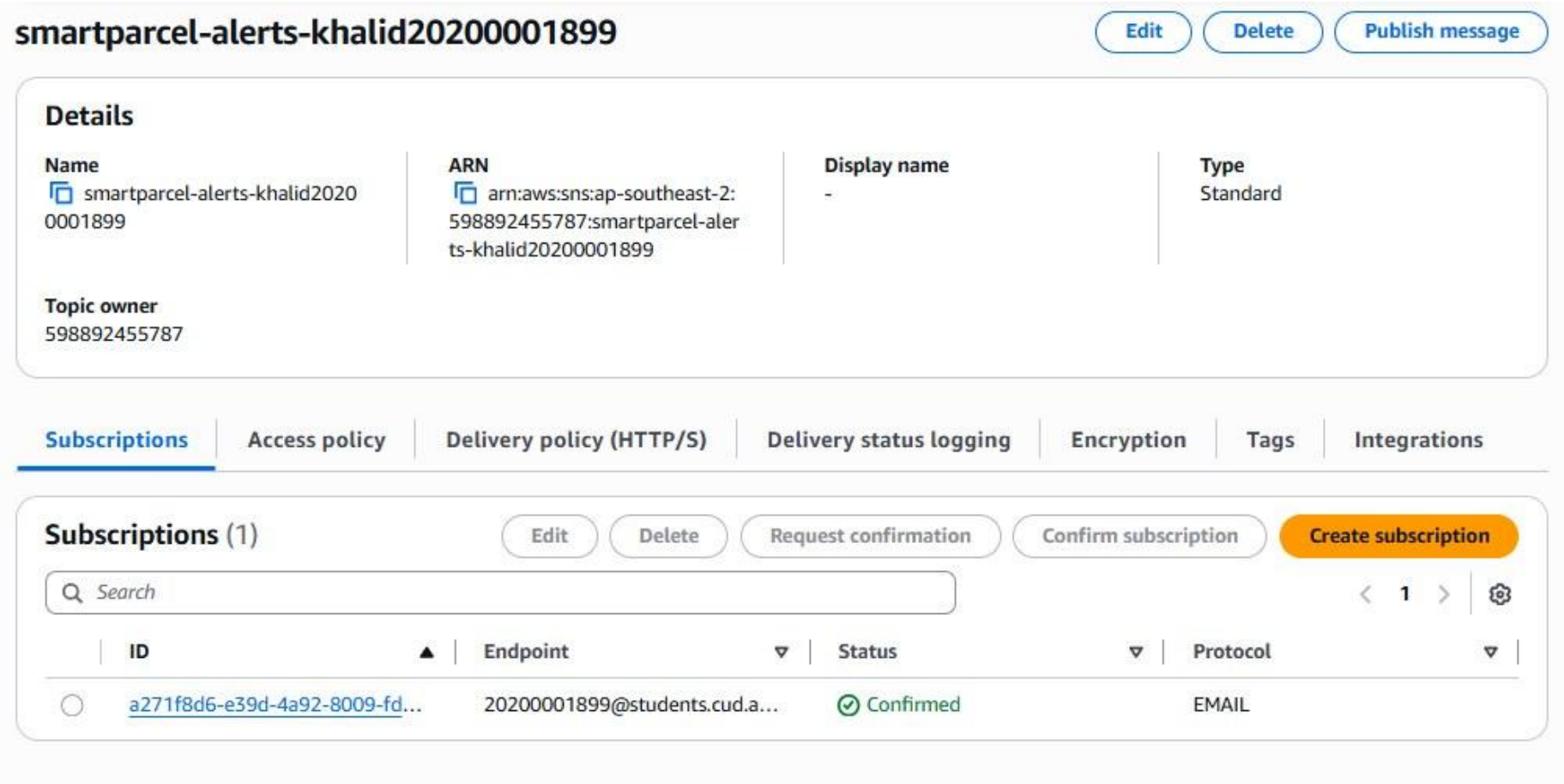


Figure 22: SNS topic

SNS topic smartparcel-alerts-khalid20200001899 was created for email alerts.

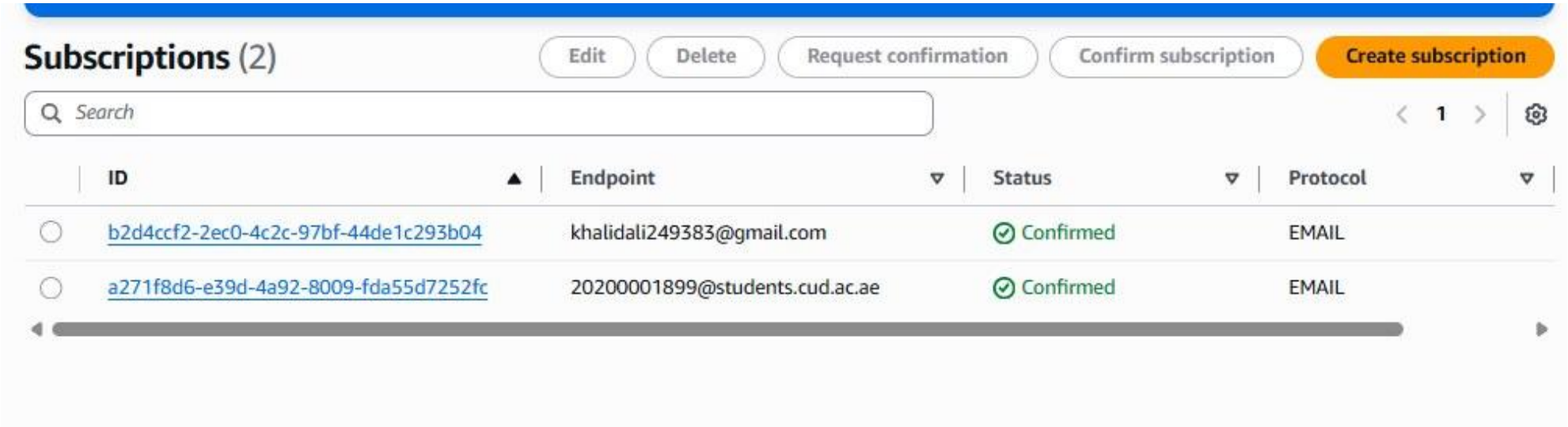


Figure 23: SNS confirmed subscriptions

The CUD email subscription is confirmed, allowing SNS to send messages.

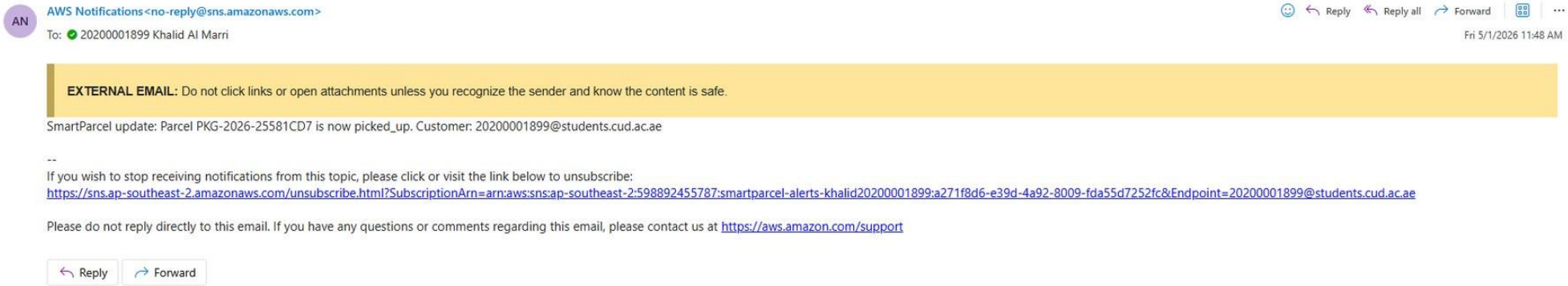


Figure 24: SNS email received after status update

The email proves the complete pipeline worked: API Gateway -> API Lambda -> SQS -> Notifier Lambda -> SNS -> Email.

3.4 DynamoDB Global Secondary Index

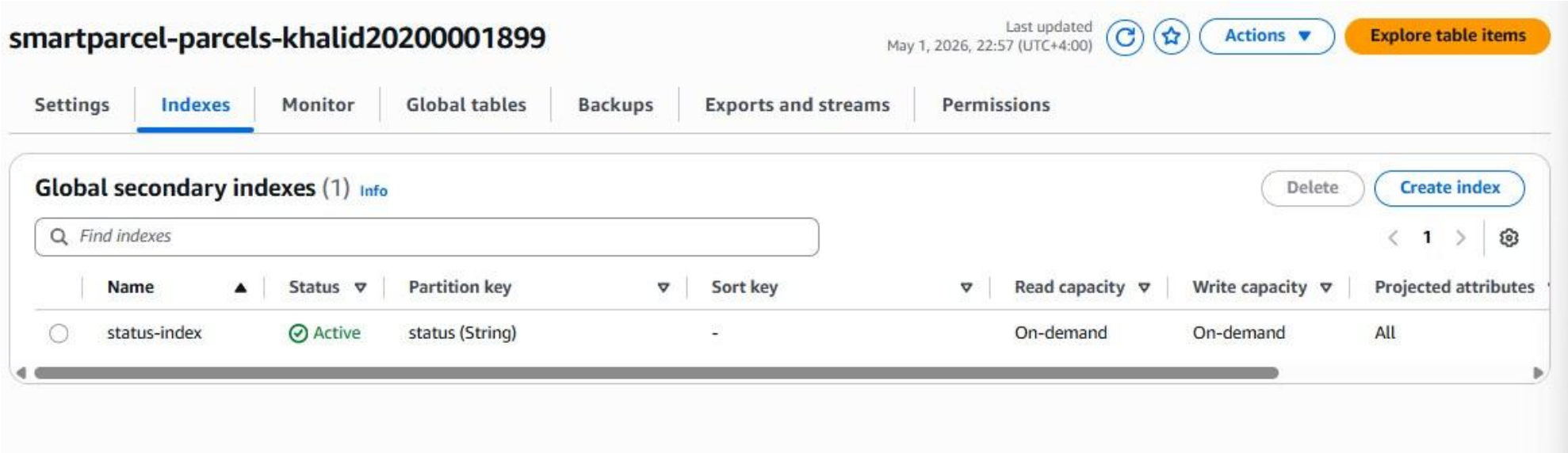


Figure 25: DynamoDB GSI status-index

status-index uses status as the partition key and projects all attributes, allowing efficient status queries.

```
> -H "x-user-role: admin" | python3 -m json.tool | tee filter_pretty.json
{
    "items": [
        {
            "receiver": "Sara",
            "parcel_id": "PKG-2026-25581CD7",
            "customer_email": "20200001899@students.cud.ac.ae",
            "updated_at": "2026-05-01T18:48:11.174538+00:00",
            "sender": "Khalid",
            "status": "picked_up",
            "created_at": "2026-05-01T18:44:24.738612+00:00",
            "address": "Dubai Mall"
        }
    ]
}
~ $
```

Figure 26: GET /parcels?status=picked_up

The status filter endpoint returned only picked_up parcels, proving the GSI-backed query works.

3.5 Concurrent Load Test

```
> if successes != 20:
>     print("\nFailed responses:")
>     for status, elapsed, text in results:
>         if status != 201:
>             print(f"Status={status}, Time={elapsed:.2f}, Error={text[:200]}")
> PY
~ $ python3 load_test.py
SmartParcel Concurrent Load Test
--------------------------------
ThreadPoolExecutor max_workers: 20
Total requests: 20
Successes: 20/20
Total time: 0.46 seconds
Average response time: 0.26 seconds
Status codes: [201, 201, 201, 201, 201, 201, 201, 201, 201, 201, 201, 201, 201, 201, 201, 201, 201, 201, 201, 201]
~ $
```

Figure 27: Load test 20 concurrent requests 20/20 success

The load test used ThreadPoolExecutor with max_workers = 20. All 20 POST requests returned 201 in 0.46 seconds, proving concurrency.

4. Phase 3 - Secure Network Architecture

4.1 VPC, Subnets, Internet Gateway, and Route Table

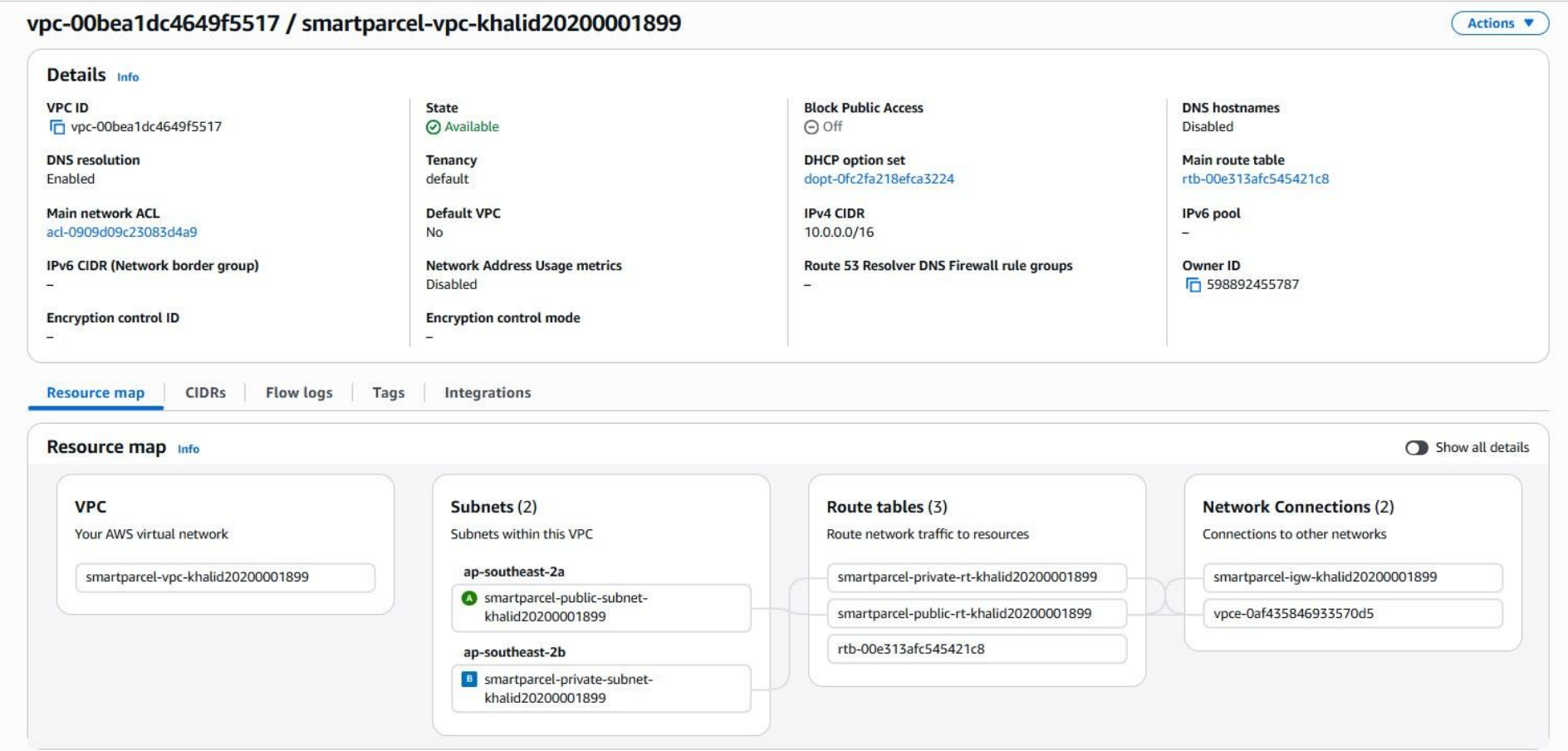


Figure 28: VPC overview and resource map

The VPC uses CIDR 10.0.0.0/16 with public and private subnets in different AZs, route tables, Internet Gateway, and a DynamoDB endpoint.

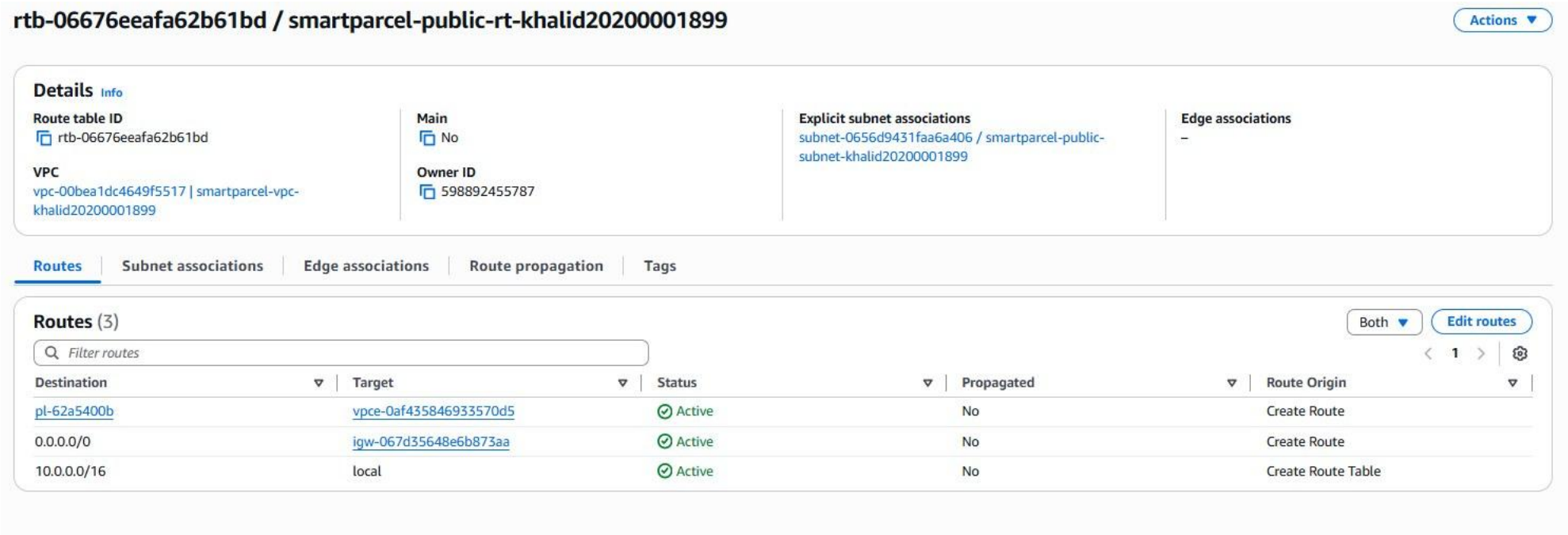


Figure 29: Public route table routes to Internet Gateway

The public route table has 0.0.0.0/0 -> Internet Gateway and endpoint route entries.

4.2 Network ACL

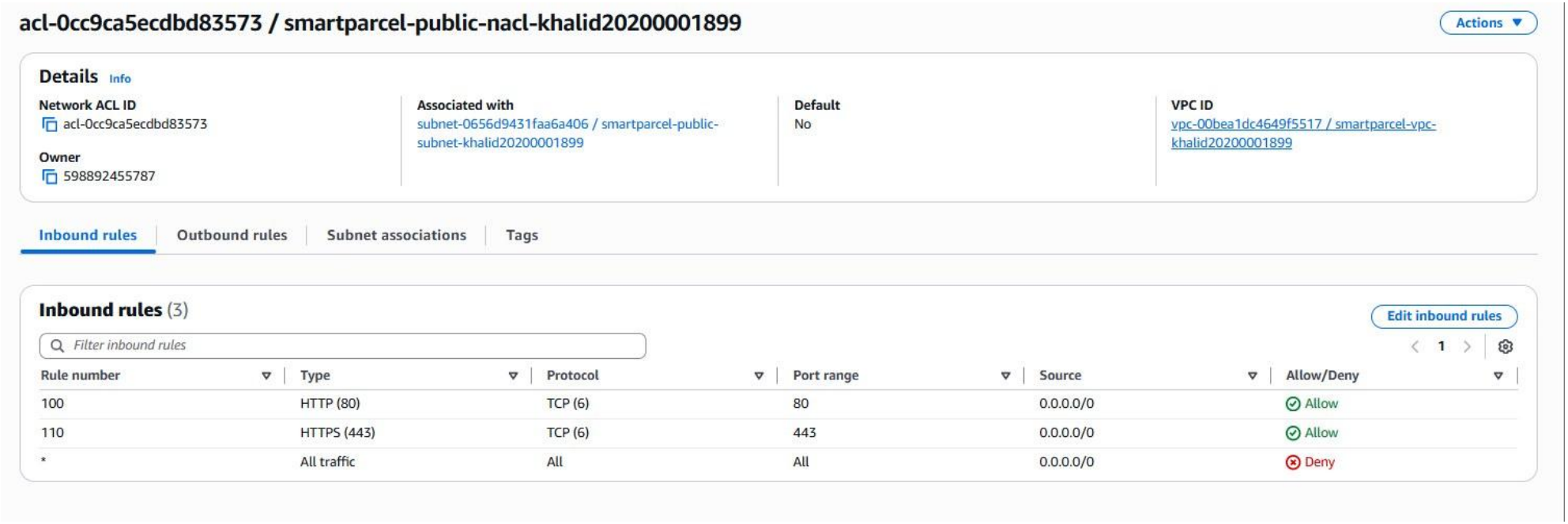


Figure 30: NACL inbound rules

Inbound rules allow HTTP 80 and HTTPS 443 from 0.0.0.0/0 and deny all else.

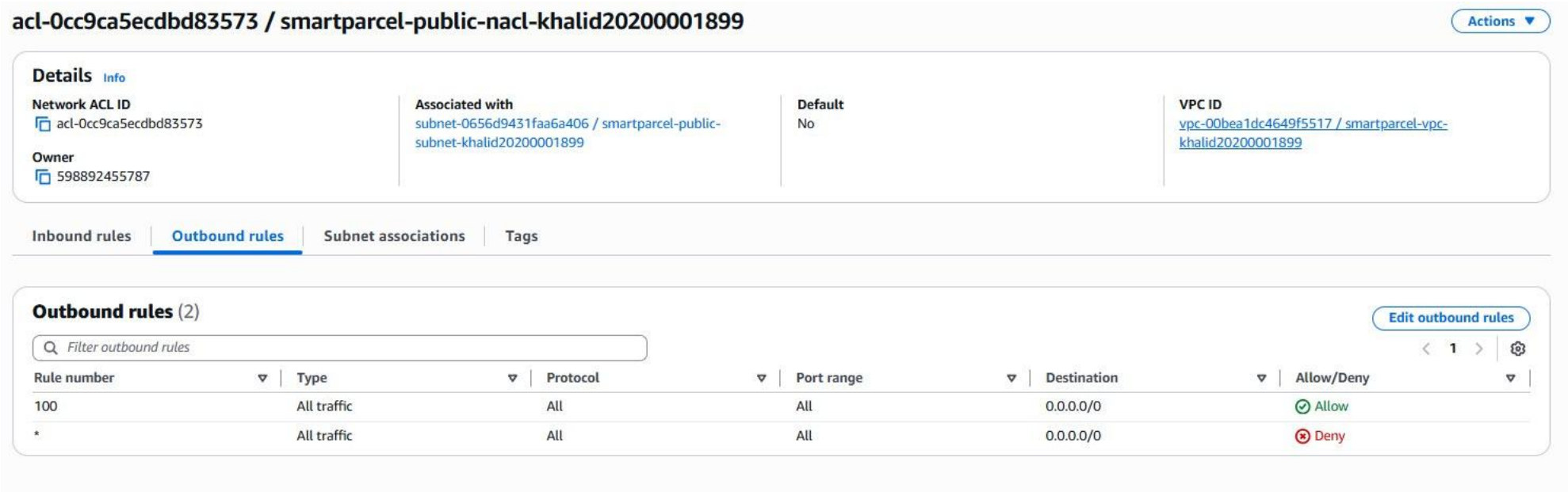


Figure 31: NACL outbound rules

Outbound rules allow all traffic. NACLs are stateless, so inbound and outbound rules are separate.

4.3 VPC Gateway Endpoint for DynamoDB

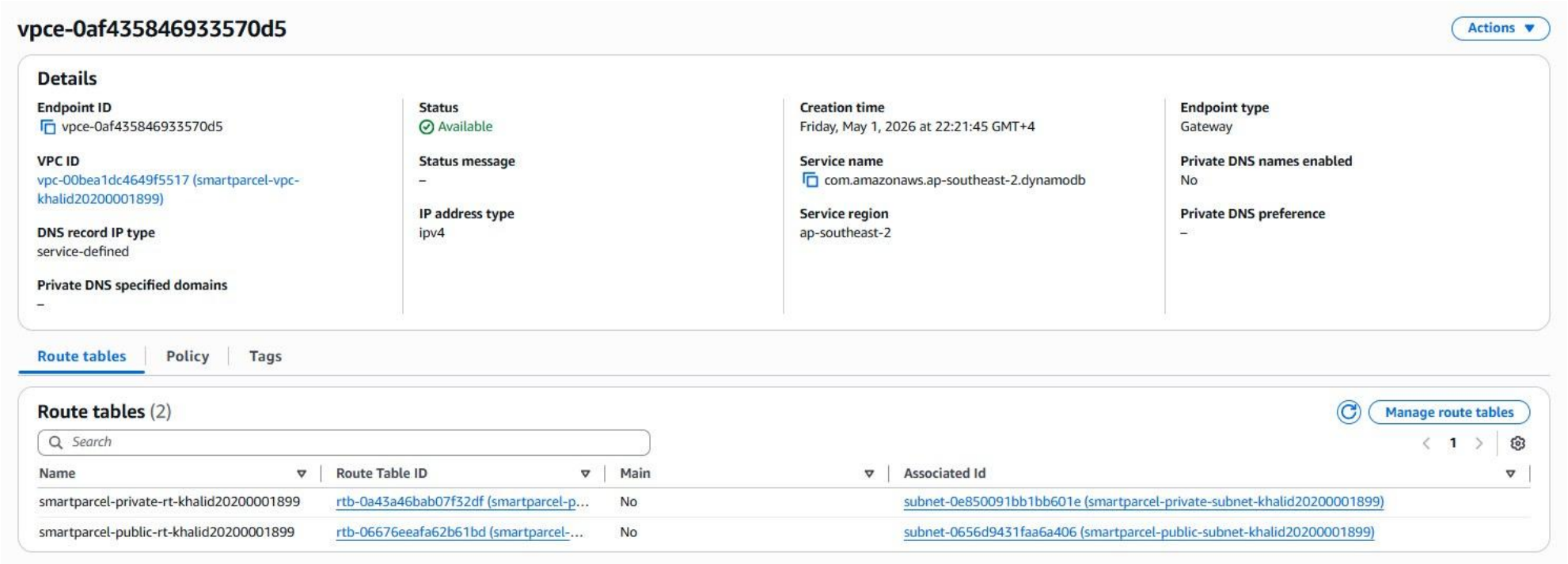


Figure 32: DynamoDB VPC Gateway Endpoint

The endpoint type is Gateway and service is com.amazonaws.ap-southeast-2.dynamodb. It is associated with the SmartParcel VPC route tables.

4.4 IAM Least Privilege

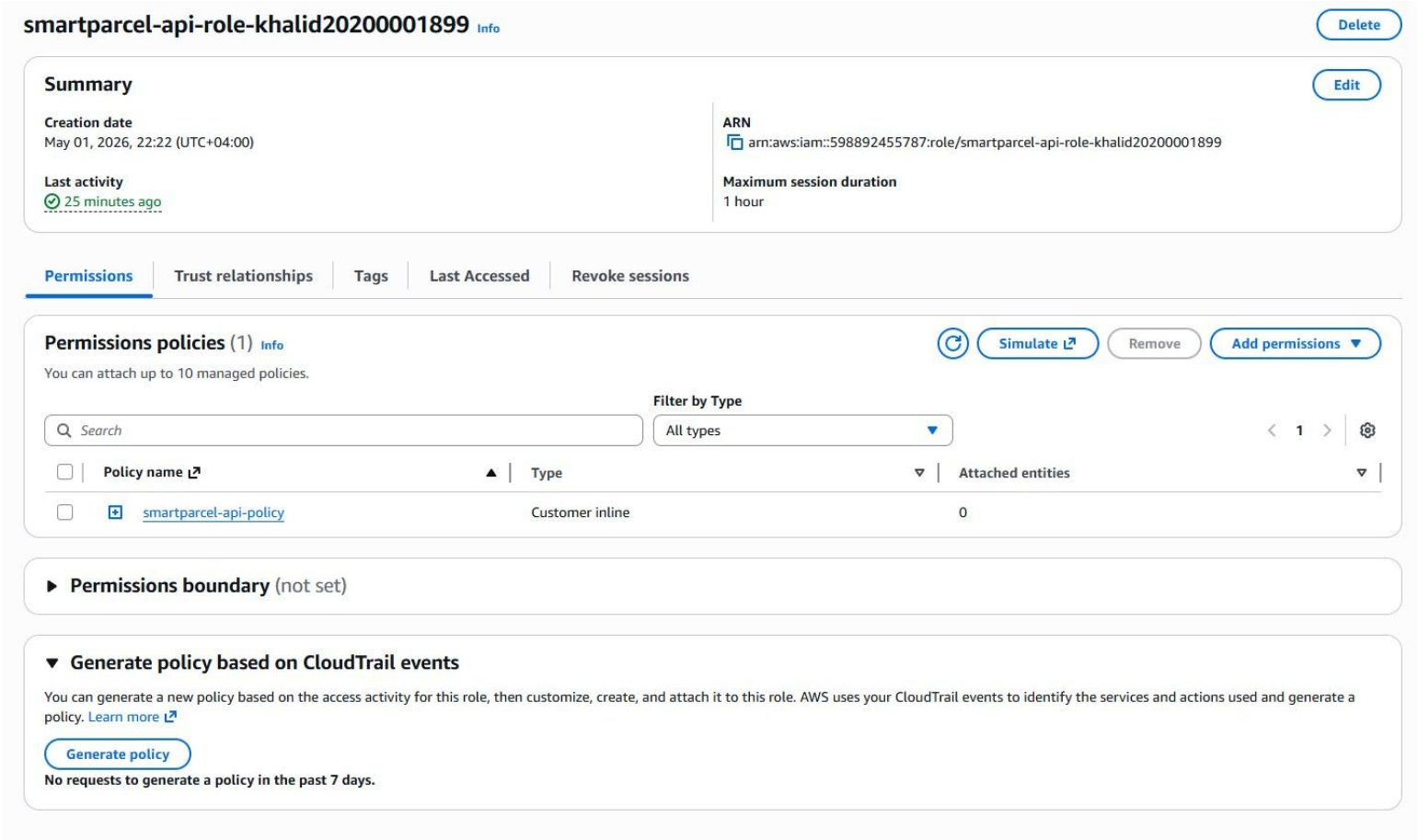


Figure 33: IAM API Lambda role

The API role uses an inline policy for only the resources required by the API Lambda.

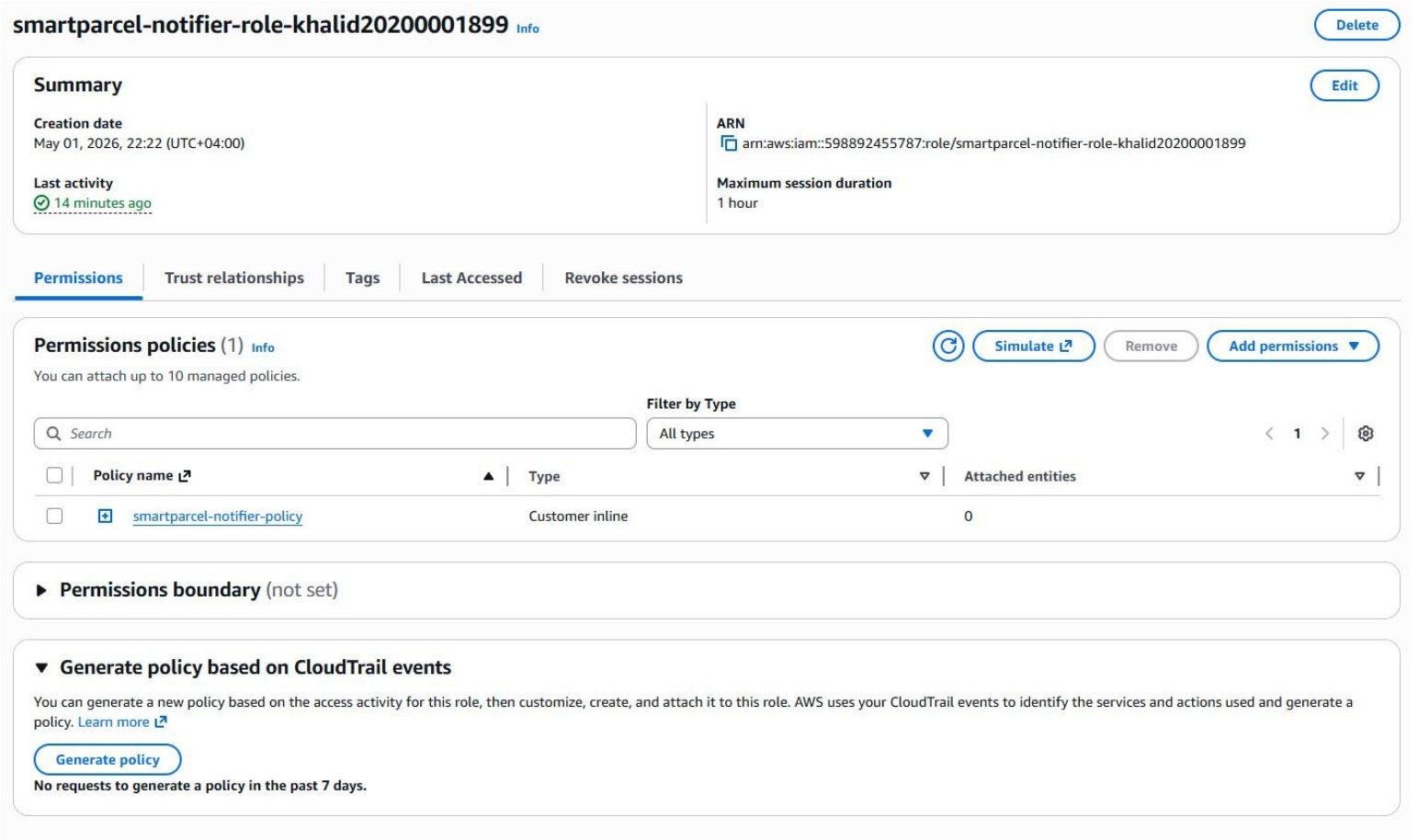


Figure 34: IAM Notifier Lambda role

The notifier role is limited to SQS receive/delete, SNS publish, and CloudWatch logging.

4.5 Input Validation and Error Handling

The API Lambda validates payload size, required fields, valid statuses, forward-only status movement, and role-based access. Invalid requests return JSON errors with correct HTTP status codes such as 400, 403, 404, and 409. The screenshots above show 403 for invalid role access and 409 for invalid cancellation.

- Payloads larger than 10 KB are rejected.
- Missing or empty fields return descriptive JSON errors.
- Status values are limited to registered, picked_up, in_transit, delivered.
- Status can only move forward.
- Only registered parcels can be cancelled.

4.6 CloudWatch Monitoring

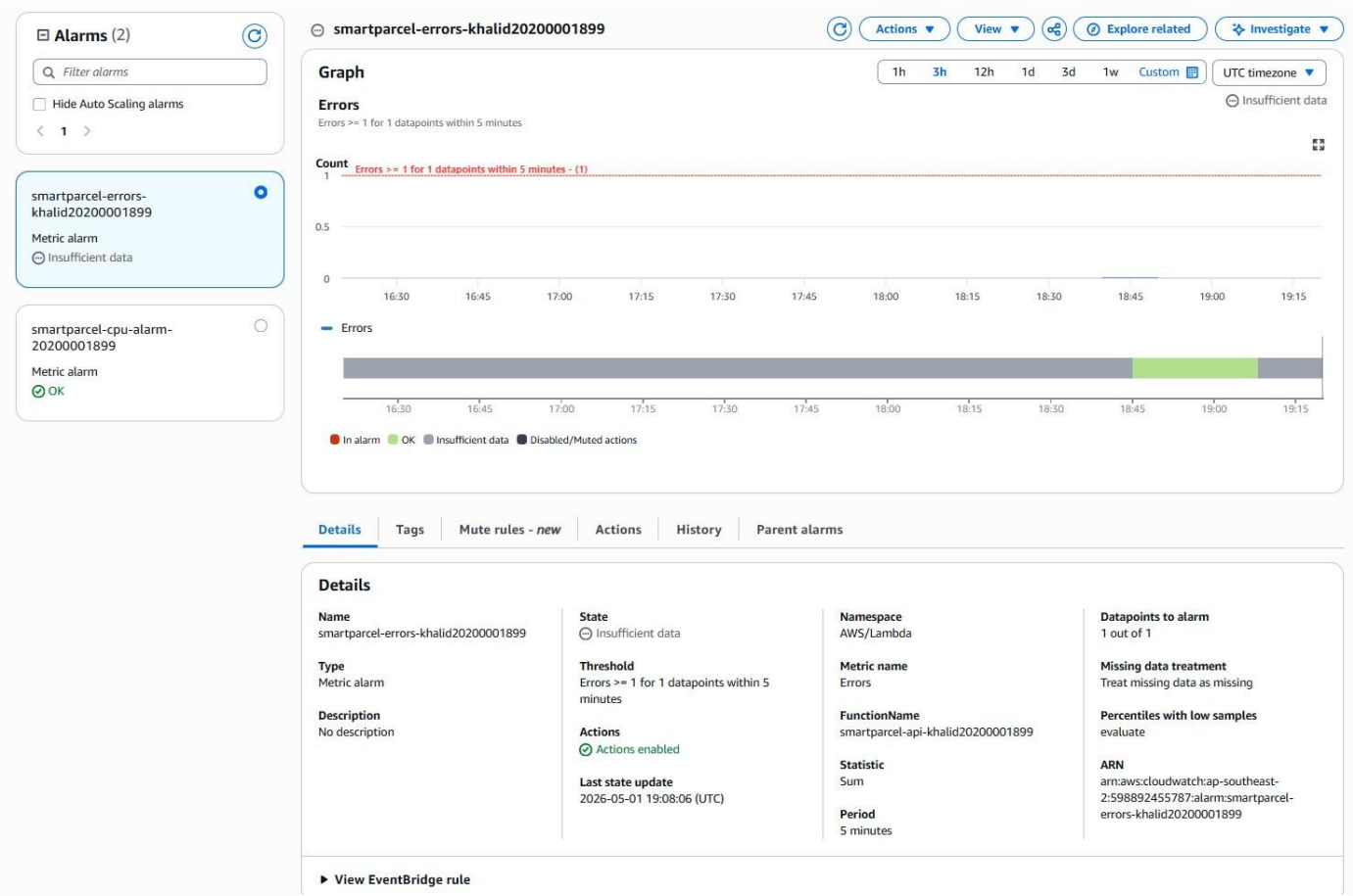


Figure 35: CloudWatch Lambda error alarm

The alarm monitors Lambda Errors >= 1 in 5 minutes and has actions enabled.

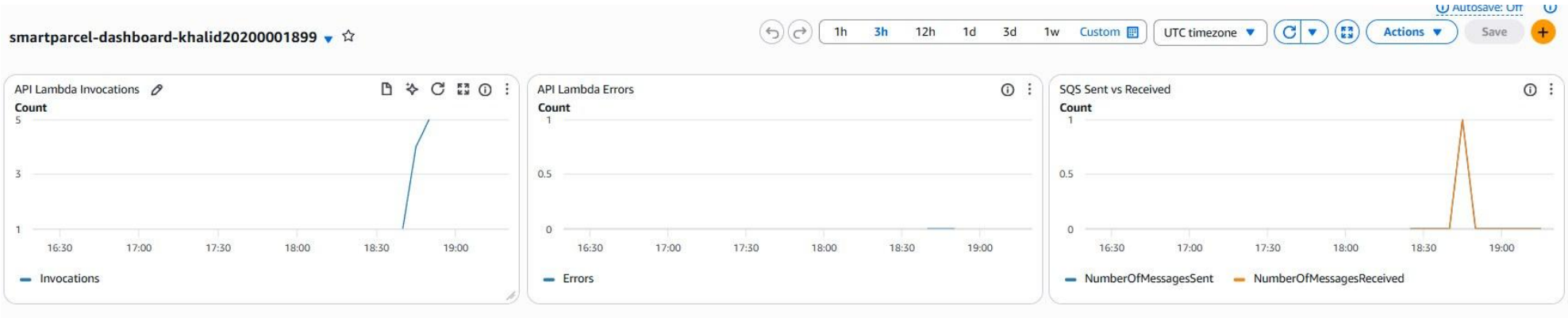


Figure 36: CloudWatch dashboard SmartParcel metrics

The dashboard shows Lambda invocations, Lambda errors, and SQS sent vs received metrics.

4.7 CloudTrail Auditing

Event history (50+) Info

Event history shows you the last 90 days of management events.

Download events | Query in Lake | Create Athena table

Lookup attributes

Event source | lambda.amazonaws.com | Filter by date and time | Clear filter | 1 2 ...

Event name	Event time	User name	Event source	Resource type	Resource name
GetFunction20150331v2	May 01, 2026, 23:08:19 (UTC+0...	root	lambda.amazonaws.com	-	-
GetFunction20150331v2	May 01, 2026, 23:08:19 (UTC+0...	root	lambda.amazonaws.com	-	-
GetFunction20150331v2	May 01, 2026, 23:08:19 (UTC+0...	root	lambda.amazonaws.com	-	-
GetFunction20150331v2	May 01, 2026, 23:08:18 (UTC+0...	root	lambda.amazonaws.com	-	-
GetFunction20150331v2	May 01, 2026, 23:08:18 (UTC+0...	root	lambda.amazonaws.com	-	-
GetFunction20150331v2	May 01, 2026, 23:08:18 (UTC+0...	root	lambda.amazonaws.com	-	-
GetAccountSettings20160819	May 01, 2026, 23:08:12 (UTC+0...	root	lambda.amazonaws.com	-	-
ListTags20170331	May 01, 2026, 23:08:12 (UTC+0...	root	lambda.amazonaws.com	-	-
ListEventSourceMappings20150331	May 01, 2026, 23:08:12 (UTC+0...	root	lambda.amazonaws.com	-	-
ListEventSourceMappings20150331	May 01, 2026, 23:08:11 (UTC+0...	root	lambda.amazonaws.com	-	-
GetPolicy20150331	May 01, 2026, 23:08:11 (UTC+0...	root	lambda.amazonaws.com	-	-
GetRuntimeManagementConfig	May 01, 2026, 23:08:11 (UTC+0...	root	lambda.amazonaws.com	-	-
GetFunctionCodeSigningConfig	May 01, 2026, 23:08:11 (UTC+0...	root	lambda.amazonaws.com	-	-
GetFunction20150331v2	May 01, 2026, 23:08:11 (UTC+0...	root	lambda.amazonaws.com	-	-
GetRuntimeManagementConfig	May 01, 2026, 23:08:11 (UTC+0...	root	lambda.amazonaws.com	-	-
GetAccountSettings20160819	May 01, 2026, 23:08:10 (UTC+0...	root	lambda.amazonaws.com	-	-
GetFunctionConfiguration20150331v2	May 01, 2026, 23:08:10 (UTC+0...	root	lambda.amazonaws.com	-	-
GetAccountSettings20160819	May 01, 2026, 23:08:10 (UTC+0...	root	lambda.amazonaws.com	-	-

Figure 37: CloudTrail Lambda event history

CloudTrail shows Lambda-related management events. CloudWatch monitors what is happening now, while CloudTrail records what happened in the past for auditing.

5. Phase 4 - Architecture and Well-Architected Review

5.1 CloudFormation Template

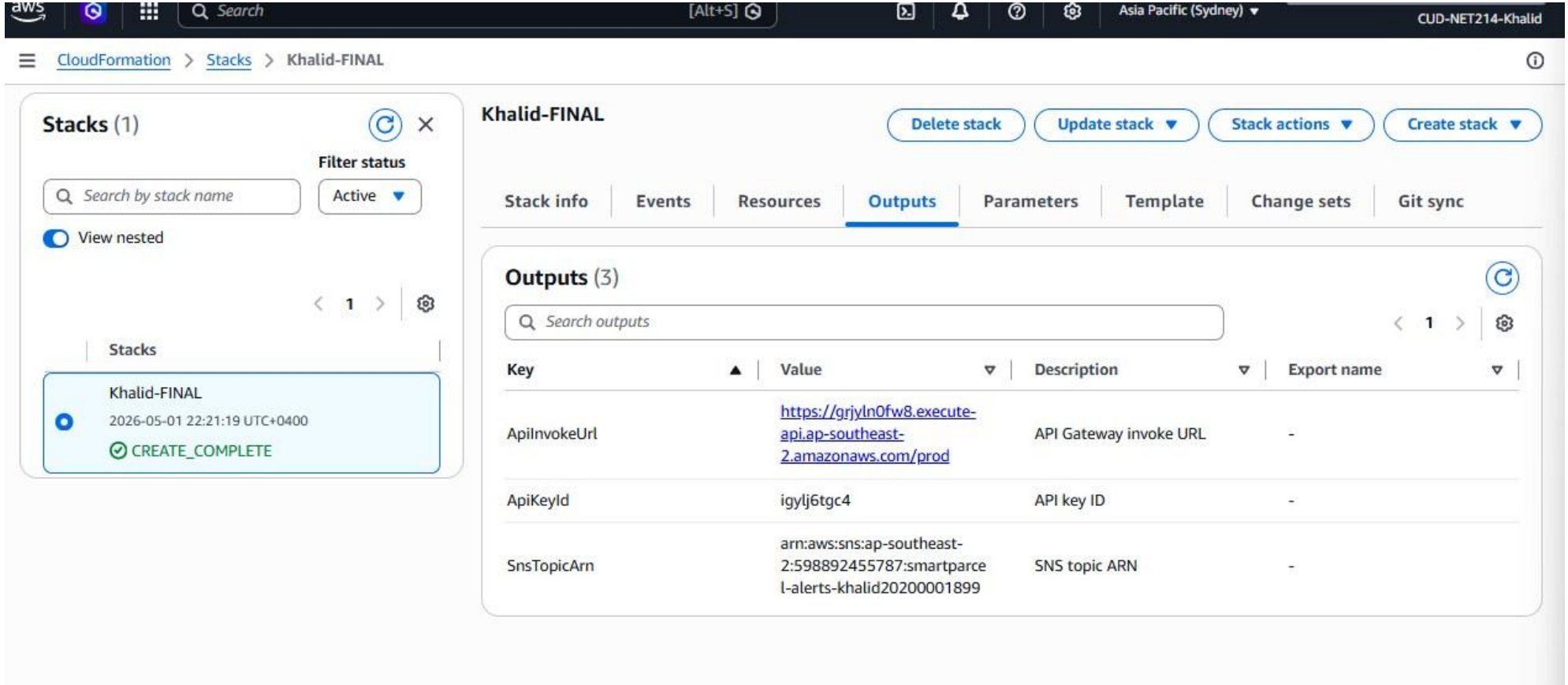


Figure 38: CloudFormation stack CREATE_COMPLETE and outputs

The Khalid-FINAL stack completed successfully and outputs the API invoke URL, API key ID, and SNS topic ARN.

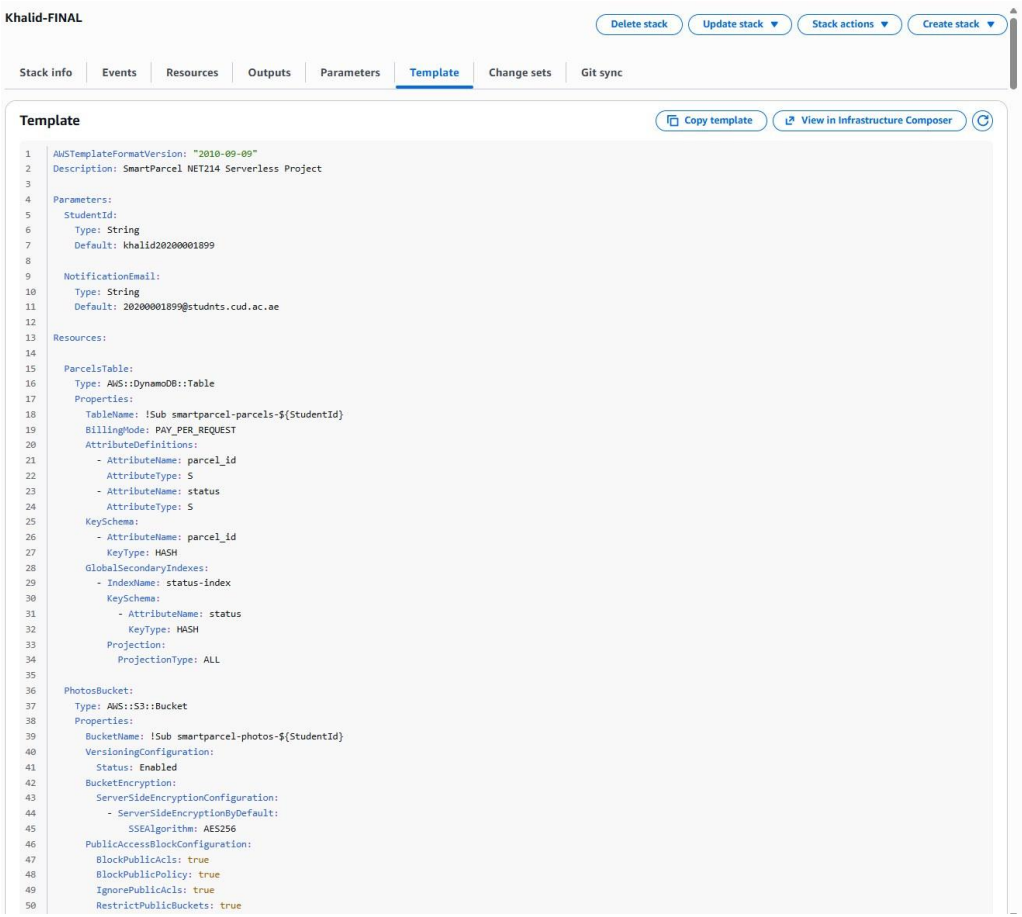
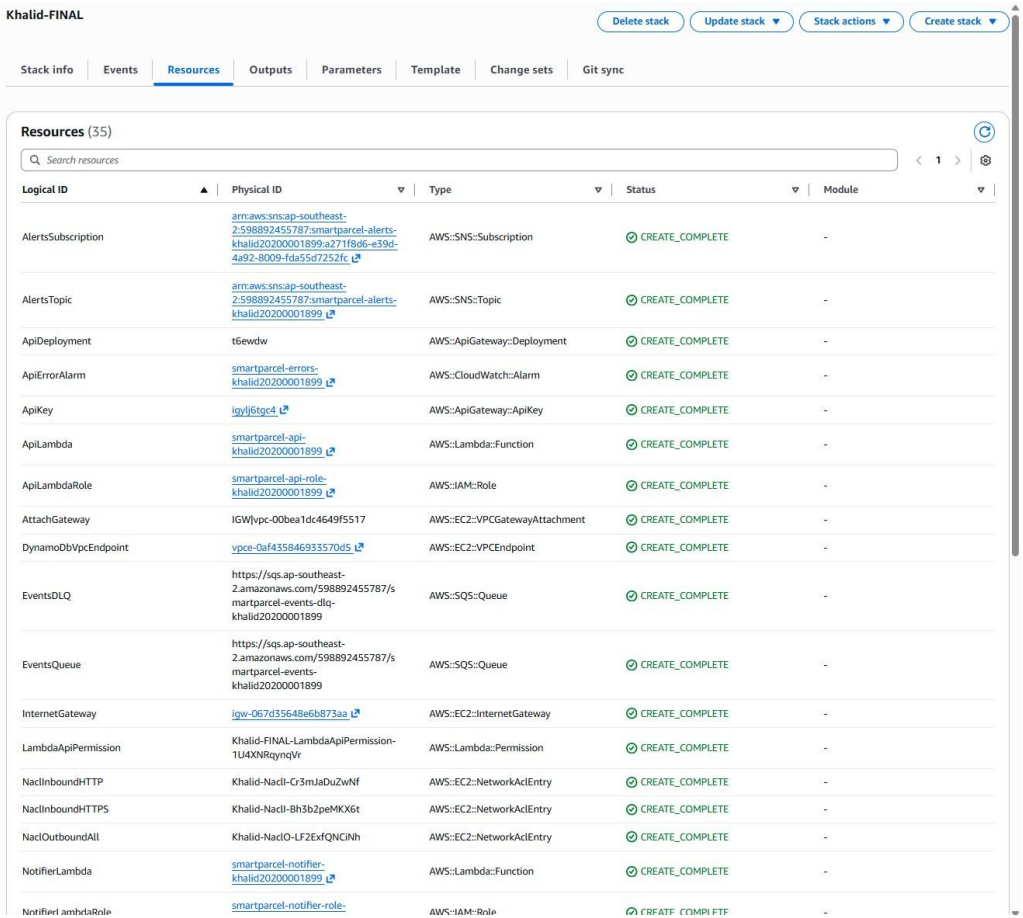


Figure 39: CloudFormation template YAML preview

The template uses StudentId and NotificationEmail parameters and defines the SmartParcel stack as Infrastructure as Code.



Khalid-FINAL

Delete stack | Update stack | Stack actions | Create stack

Stack info | Events | Resources | Outputs | Parameters | Template | Change sets | Git sync

Resources (35)

Search resources

Logical ID	Physical ID	Type	Status	Module
AlertsSubscription	arn:aws:sns:ap-southeast-2:598892455787:smartparcel-alerts-khalid20200001899:a271f8d6-e39d-4a92-8009-fda55d7252fc	AWS::SNS::Subscription	CREATE_COMPLETE	-
AlertsTopic	arn:aws:sns:ap-southeast-2:598892455787:smartparcel-alerts-khalid20200001899	AWS::SNS::Topic	CREATE_COMPLETE	-
ApiDeployment	t6ewdw	AWS::ApiGateway::Deployment	CREATE_COMPLETE	-
ApiErrorAlarm	smartparcel-errors-khalid20200001899	AWS::CloudWatch::Alarm	CREATE_COMPLETE	-
ApiKey	igylj6tgc4	AWS::ApiGateway::ApiKey	CREATE_COMPLETE	-
ApiLambda	smartparcel-api-khalid20200001899	AWS::Lambda::Function	CREATE_COMPLETE	-
ApiLambdaRole	smartparcel-api-role-khalid20200001899	AWS::IAM::Role	CREATE_COMPLETE	-
AttachGateway	IGW|vpc-00bea1dc4649f5517	AWS::EC2::VPCGatewayAttachment	CREATE_COMPLETE	-
DynamoDbVpcEndpoint	vpce-0af435846933570d5	AWS::EC2::VPCEndpoint	CREATE_COMPLETE	-
EventsDLQ	https://sqs.ap-southeast-2.amazonaws.com/598892455787/smartparcel-events-dlq-khalid20200001899	AWS::SQS::Queue	CREATE_COMPLETE	-
EventsQueue	https://sqs.ap-southeast-2.amazonaws.com/598892455787/smartparcel-events-khalid20200001899	AWS::SQS::Queue	CREATE_COMPLETE	-
InternetGateway	igw-067d35648e6b873aa	AWS::EC2::InternetGateway	CREATE_COMPLETE	-
LambdaApiPermission	Khalid-FINAL-LambdaApiPermission-1U4XNRqynqVr	AWS::Lambda::Permission	CREATE_COMPLETE	-
NaclInboundHTTP	Khalid-NaclI-Cr3mJaDuZwNf	AWS::EC2::NetworkAclEntry	CREATE_COMPLETE	-
NaclInboundHTTPS	Khalid-NaclI-Bh3b2peMKX6t	AWS::EC2::NetworkAclEntry	CREATE_COMPLETE	-
NaclOutboundAll	Khalid-NaclO-LF2ExfQNCiNh	AWS::EC2::NetworkAclEntry	CREATE_COMPLETE	-
NotifierLambda	smartparcel-notifier-khalid20200001899	AWS::Lambda::Function	CREATE_COMPLETE	-
NotifierLambdaRole	smartparcel-notifier-role-	AWS::IAM::Role	CREATE_COMPLETE	-

Figure 40: CloudFormation resources created - top

The stack created resources including SNS, API Gateway deployment, CloudWatch alarm, API key, Lambda API, IAM role, endpoint, SQS queues, IGW, and NACL entries.

NotifierLambdaRole	smartparcel-notifier-role-khalid20200001899	AWS::IAM::Role	CREATE_COMPLETE	-
ParcelsTable	smartparcel-parcels-khalid20200001899	AWS::DynamoDB::Table	CREATE_COMPLETE	-
PhotosBucket	smartparcel-photos-khalid20200001899	AWS::S3::Bucket	CREATE_COMPLETE	-
PrivateRouteTable	rtb-0a43a46bab07f32df	AWS::EC2::RouteTable	CREATE_COMPLETE	-
PrivateSubnet	subnet-0e850091bb1bb601e	AWS::EC2::Subnet	CREATE_COMPLETE	-
PrivateSubnetAssociation	rtbassoc-07b991715bbad93e7	AWS::EC2::SubnetRouteTableAssociation	CREATE_COMPLETE	-
PublicNetworkAcl	acl-0cc9ca5ecdbd83573	AWS::EC2::NetworkAcl	CREATE_COMPLETE	-
PublicRoute	rtb-06676eeafa62b61bd|0.0.0.0/0	AWS::EC2::Route	CREATE_COMPLETE	-
PublicRouteTable	rtb-06676eeafa62b61bd	AWS::EC2::RouteTable	CREATE_COMPLETE	-
PublicSubnet	subnet-0656d9431faa6a406	AWS::EC2::Subnet	CREATE_COMPLETE	-
PublicSubnetAssociation	rtbassoc-0f189f5de3f6d6ac9	AWS::EC2::SubnetRouteTableAssociation	CREATE_COMPLETE	-
PublicSubnetNaclAssociation	aclassoc-099d9ce5d5ec30db3	AWS::EC2::SubnetNetworkAclAssociation	CREATE_COMPLETE	-
SmartParcelApi	grjyln0fw8	AWS::ApiGateway::RestApi	CREATE_COMPLETE	-
SmartParcelDashboard	smartparcel-dashboard-khalid20200001899	AWS::CloudWatch::Dashboard	CREATE_COMPLETE	-
SmartParcelVpc	vpc-00bea1dc4649f5517	AWS::EC2::VPC	CREATE_COMPLETE	-
SqsTrigger	2b608f96-3eda-4f16-bce9-3fff3de1ca2c	AWS::Lambda::EventSourceMapping	CREATE_COMPLETE	-
UsagePlan	m0ojd5	AWS::ApiGateway::UsagePlan	CREATE_COMPLETE	-
UsagePlanKey	igylj6tgc4:m0ojd5	AWS::ApiGateway::UsagePlanKey	CREATE_COMPLETE	-

Figure 41: CloudFormation resources created - bottom

The stack created the DynamoDB table, S3 bucket, subnets, route tables, REST API, dashboard, VPC, SQS trigger, usage plan, and usage plan key.

5.2 Architecture Document - Network Design

VPC CIDR: 10.0.0.0/16. Public subnet: 10.0.1.0/24 in ap-southeast-2a. Private subnet: 10.0.2.0/24 in ap-southeast-2b. The public route table points 0.0.0.0/0 to the Internet Gateway. A Gateway VPC Endpoint for DynamoDB is associated with the route tables.

The public subnet is for resources that require a public route. The private subnet is for internal resources. API Gateway provides the public HTTPS endpoint, so Lambda does not need an Internet Gateway. SmartParcel would need a NAT Gateway only if private subnet resources such as EC2 needed outbound internet access, but it was intentionally avoided to prevent cost.

For a custom domain, I would use Route 53 with API Gateway custom domain and ACM certificate. Simple routing is enough for one regional endpoint. Latency-based routing would be used if SmartParcel expanded to multiple regions.

5.3 Architecture Document - Security Design

Shared Responsibility Model: AWS secures the cloud infrastructure, data centers, and managed service platform. I secured the configurations: IAM roles, API key, Lambda role checks, encryption, S3 public access block, CloudWatch alarms, CloudTrail auditing, and VPC resources.

Encryption at rest: DynamoDB uses AWS-managed encryption and S3 uses SSE-S3. SSE-S3 was chosen because it is simple and AWS-managed. SSE-KMS would be chosen for customer-managed keys, key rotation control, cross-account permissions, or detailed key auditing. SSE-C would require customer-managed keys and is more operationally difficult.

Encryption in transit: API Gateway uses HTTPS/TLS. Access control uses x-api-key at API Gateway and X-User-Role checks inside Lambda. IAM roles follow least privilege.

5.4 High Availability and Disaster Recovery

Lambda, DynamoDB, SQS, and SNS are managed AWS services designed for availability inside the region. Lambda runs across multiple Availability Zones, DynamoDB is replicated within the region, and SQS stores messages durably until processed.

For 10x traffic, I would monitor Lambda concurrency, API Gateway throttling, DynamoDB usage, and SQS backlog. DynamoDB on-demand is suitable for unpredictable workloads. If EC2 was required, I would use an ALB across two public subnets and an Auto Scaling Group across two private subnets, with general purpose t3/t4g instances and gp3 EBS volumes.

DR strategy: Backup-restore is the cheapest option for this project. Production improvements would include DynamoDB point-in-time recovery, S3 Cross-Region Replication, and DynamoDB Global Tables. Warm standby and active-active multi-site provide faster recovery but cost more.

5.5 Database Choice

DynamoDB fits SmartParcel because parcel records are simple key-value documents, the main lookup is by parcel_id, and the status-index supports status filtering. RDS would be better if the system needed complex joins, relational reports, or multi-table transactions.

5.6 Cost Analysis

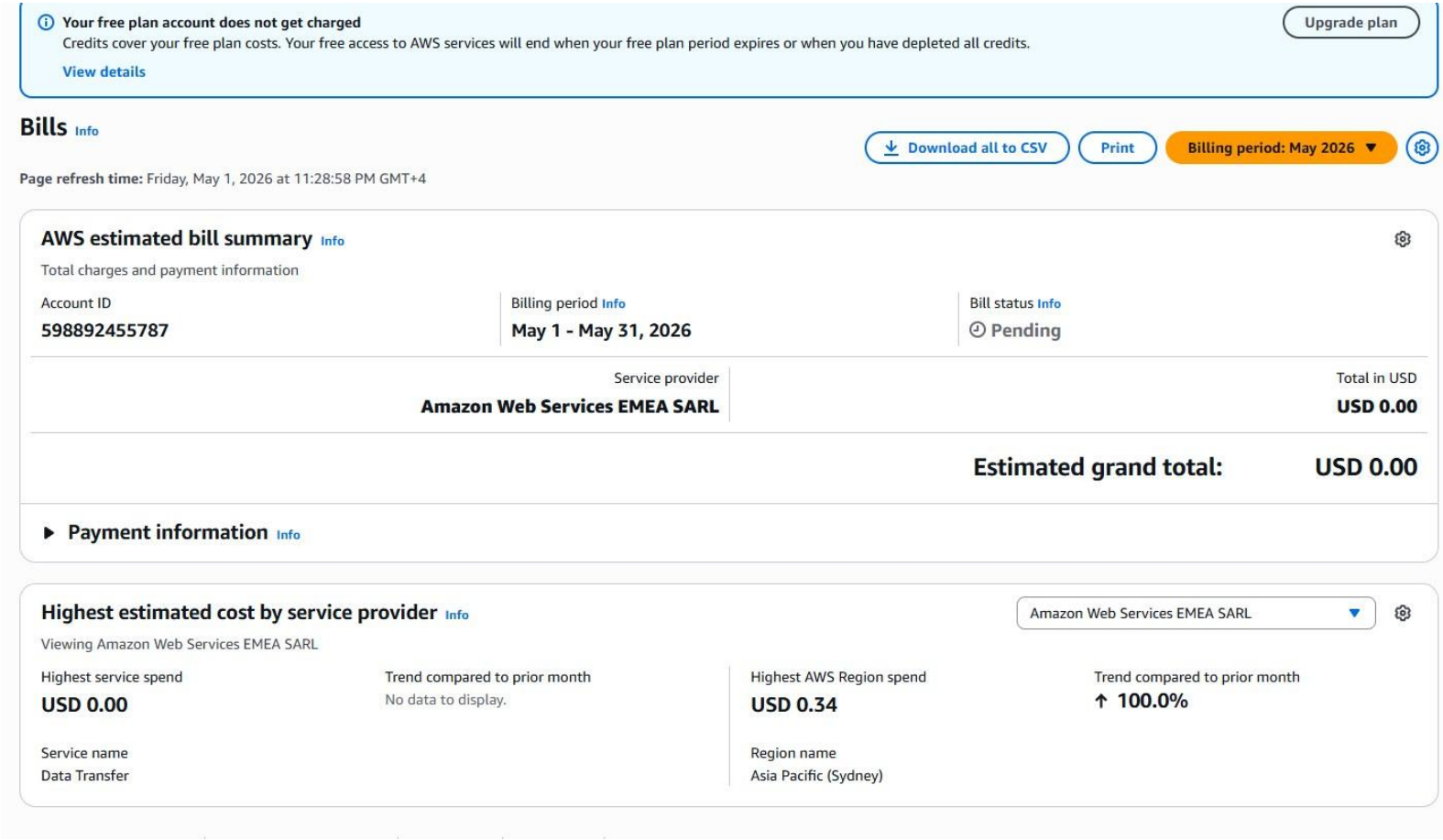


Figure 42: Billing screenshot showing USD 0.00

The billing page shows estimated grand total USD 0.00. The design avoids paid resources such as NAT Gateway, ALB, RDS, EC2, and Elastic IP.

Service	Current usage	Cost	10x usage	10x note
Lambda	Low test requests	~$0.00	10x requests	Still low; pay per request
API Gateway	REST calls	~$0.00	10x calls	Small per-call cost
DynamoDB	On-demand low usage	~$0.00	Higher read/write	Scales with usage
S3	Small storage	~$0.00	More photos	Lifecycle reduces cost
SQS	Low messages	~$0.00	More messages	Low request cost
SNS	Email notifications	~$0.00	More emails	Low notification cost

Lambda is better than EC2 for this workload because it has no idle server cost. If SmartParcel ran on EC2 with stable traffic, Reserved Instances or Savings Plans would reduce cost. Spot is cheap but interruptible, so it is not ideal for the main production API.

5.7 Well-Architected Framework Review

Operational Excellence

What SmartParcel does well: CloudWatch dashboards, alarms, CloudTrail, and CloudFormation support monitoring and repeatable deployment.

Future improvement: Add CI/CD with GitHub Actions or AWS CodePipeline.

Security

What SmartParcel does well: The design uses API keys, role-based checks, IAM least privilege, encryption, S3 public blocking, and CloudTrail.

Future improvement: Replace role header with Cognito/JWT authorizer.

Reliability

What SmartParcel does well: SQS decouples notifications, DLQ captures repeated failures, and managed services improve availability.

Future improvement: Enable DynamoDB point-in-time recovery and DLQ alarms.

Performance Efficiency

What SmartParcel does well: Serverless services scale automatically, and DynamoDB on-demand plus GSI supports efficient access.

Future improvement: Add caching for repeated admin/status queries and tune API throttling.

Cost Optimization

What SmartParcel does well: Serverless pay-per-use avoids EC2 cost, S3 lifecycle reduces storage cost, and NAT Gateway was avoided.

Future improvement: Set log retention and review Cost Explorer monthly.

6. Conclusion

SmartParcel was successfully deployed as a serverless AWS application. It supports parcel creation, parcel retrieval, admin listing, status updates, API key authentication, role-based access control, event-driven notifications, monitoring, auditing, secure storage, and Infrastructure as Code through CloudFormation.

The screenshots and tests show the deployed system meets the main requirements: all endpoints tested, SQS/SNS pipeline working, 20/20 concurrent load test success, VPC and security resources created, and billing remaining at USD 0.00.

Appendix A - Lambda API Code

```
import json, os, uuid
import boto3
from boto3.dynamodb.conditions import Key
from datetime import datetime, timezone

dynamodb = boto3.resource("dynamodb")
sqs = boto3.client("sqs")

TABLE_NAME = os.environ["TABLE_NAME"]
QUEUE_URL = os.environ["QUEUE_URL"]
table = dynamodb.Table(TABLE_NAME)

VALID_STATUSES = ["registered", "picked_up", "in_transit", "delivered"]

def response(code, body):
    return {
        "statusCode": code,
        "headers": {"Content-Type": "application/json"},
        "body": json.dumps(body)
    }

def parse_body(event):
    body = event.get("body") or "{}"
    if len(body.encode("utf-8")) > 10240:
        raise ValueError("Payload too large. Maximum allowed size is 10 KB.")
    try:
        return json.loads(body)
    except:
        raise ValueError("Invalid JSON body")

def role(event):
    headers = event.get("headers") or {}
    return headers.get("X-User-Role") or headers.get("x-user-role") or ""

def require_role(event, allowed):
    user_role = role(event)
    if user_role not in allowed:
        return False
    return True

def require_fields(body, fields):
    for field in fields:
        if field not in body or body[field] in ["", None]:
            raise ValueError(f"Missing field: {field}")

def lambda_handler(event, context):
    try:
        method = event.get("httpMethod")
        path = event.get("resource")
        params = event.get("pathParameters") or {}

        if method == "POST" and path == "/parcels":
            if not require_role(event, ["driver", "admin"]):
                return response(403, {"error": "Forbidden"})
            return create_parcel(event)

        if method == "GET" and path == "/parcels/{id}":
            if not require_role(event, ["driver", "customer", "admin"]):
                return response(403, {"error": "Forbidden"})
            return get_parcel(params.get("id"))

        if method == "GET" and path == "/parcels":
            if not require_role(event, ["admin"]):
                return response(403, {"error": "Forbidden"})
            return list_parcels(event)

        if method == "PUT" and path == "/parcels/{id}/status":
            if not require_role(event, ["driver"]):
                return response(403, {"error": "Forbidden"})
            return update_status(event, params.get("id"))

        if method == "DELETE" and path == "/parcels/{id}":
            if not require_role(event, ["admin"]):
                return response(403, {"error": "Forbidden"})
            return delete_parcel(params.get("id"))

        return response(404, {"error": "Not found"})

    except ValueError as e:
        return response(400, {"error": str(e)})
    except Exception as e:
        print("ERROR:", str(e))
        return response(500, {"error": "Internal server error"})

def create_parcel(event):
    body = parse_body(event)
    require_fields(body, ["sender", "receiver", "address", "email"])
```

```
    parcel_id = "PKG-" + datetime.now().strftime("%Y") + "-" + uuid.uuid4().hex[:8].upper()
    now = datetime.now(timezone.utc).isoformat()

    item = {
        "parcel_id": parcel_id,
        "sender": body["sender"],
        "receiver": body["receiver"],
        "address": body["address"],
        "customer_email": body["email"],
        "status": "registered",
        "created_at": now,
        "updated_at": now
    }

    table.put_item(Item=item)
    return response(201, item)

def get_parcel(parcel_id):
    result = table.get_item(Key={"parcel_id": parcel_id})
    item = result.get("Item")
    if not item:
        return response(404, {"error": "Parcel not found"})
    return response(200, item)

def list_parcels(event):
    query = event.get("queryStringParameters") or {}
    status = query.get("status")

    if status:
        if status not in VALID_STATUSES:
            return response(400, {"error": "Invalid status"})
        result = table.query(
            IndexName="status-index",
            KeyConditionExpression=Key("status").eq(status)
        )
    else:
        result = table.scan()

    return response(200, {"items": result.get("Items", [])})

def update_status(event, parcel_id):
    body = parse_body(event)
    # Rubric/lab uses "new_status". "status" is also accepted to avoid test mistakes.
    new_status = body.get("new_status") or body.get("status")
    if not new_status:
        raise ValueError("Missing field: new_status")
    if new_status not in VALID_STATUSES:
        return response(400, {"error": "Invalid status"})

    result = table.get_item(Key={"parcel_id": parcel_id})
    item = result.get("Item")
    if not item:
        return response(404, {"error": "Parcel not found"})

    old_status = item["status"]
    if VALID_STATUSES.index(new_status) <= VALID_STATUSES.index(old_status):
        return response(409, {"error": "Status can only move forward"})

    now = datetime.now(timezone.utc).isoformat()

    table.update_item(
        Key={"parcel_id": parcel_id},
        UpdateExpression="SET #s = :s, updated_at = :u",
        ExpressionAttributeNames={"#s": "status"},
        ExpressionAttributeValues={":s": new_status, ":u": now}
    )

    sqs.send_message(
        QueueUrl=QUEUE_URL,
        MessageBody=json.dumps({
            "parcel_id": parcel_id,
            "new_status": new_status,
            "customer_email": item["customer_email"],
            "timestamp": now
        })
    )

    item["status"] = new_status
    item["updated_at"] = now
    return response(200, item)

def delete_parcel(parcel_id):
    result = table.get_item(Key={"parcel_id": parcel_id})
    item = result.get("Item")
    if not item:
        return response(404, {"error": "Parcel not found"})

    if item["status"] != "registered":
        return response(409, {"error": "Only registered parcels can be cancelled"})
```

```
table.delete_item(Key={"parcel_id": parcel_id})
return response(200, {"message": "Parcel cancelled", "parcel_id": parcel_id})
```

Appendix B - Lambda Notifier Code

```
import json, os
import boto3

sns = boto3.client("sns")
TOPIC_ARN = os.environ["TOPIC_ARN"]

def lambda_handler(event, context):
    for record in event.get("Records", []):
        try:
            body = json.loads(record["body"])
            parcel_id = body["parcel_id"]
            new_status = body["new_status"]
            customer_email = body.get("customer_email", "customer")

            message = f"SmartParcel update: Parcel {parcel_id} is now {new_status}. Customer: {customer_email}"

            sns.publish(
                TopicArn=TOPIC_ARN,
                Subject="SmartParcel Status Update",
                Message=message
            )

        except Exception as e:
            print("Malformed message skipped:", str(e))
            continue

    return {"statusCode": 200}
```

Appendix C - load_test.py

```python
"""SmartParcel concurrent load test - Khalid 20200001899
Replace API_URL and API_KEY before running in AWS CloudShell.
"""
import json
import time
import urllib.request
from concurrent.futures import ThreadPoolExecutor

API_URL = "https://YOUR_API_ID.execute-api.ap-southeast-2.amazonaws.com/prod"
API_KEY = "PASTE_API_KEY_VALUE_HERE"


def create_parcel(i):
    start = time.time()
    try:
        body = json.dumps({
            "sender": f"Driver-{i}",
            "receiver": f"Khalid-Customer-{i}",
            "address": f"Dubai Test Address {i}",
            "email": "20200001899@students.cud.ac.ae"
        }).encode("utf-8")

        req = urllib.request.Request(
            f"{API_URL}/parcels",
            data=body,
            headers={
                "Content-Type": "application/json",
                "x-api-key": API_KEY,
                "X-User-Role": "driver"
            },
            method="POST"
        )

        with urllib.request.urlopen(req, timeout=20) as resp:
            data = json.loads(resp.read().decode("utf-8"))
            return {
                "id": i,
                "success": resp.status == 201,
                "status_code": resp.status,
                "parcel_id": data.get("parcel_id", ""),
                "time": round(time.time() - start, 3)
            }
    except Exception as e:
        return {
            "id": i,
            "success": False,
            "status_code": "ERROR",
            "error": str(e),
            "time": round(time.time() - start, 3)
        }


def main():
    total_requests = 20
    start = time.time()
    print(f"Load Test - {total_requests} concurrent POST /parcels requests")

    with ThreadPoolExecutor(max_workers=20) as pool:
        results = list(pool.map(create_parcel, range(1, total_requests + 1)))

    wall_clock = round(time.time() - start, 3)
    success_count = sum(1 for r in results if r["success"])
    avg_response_time = round(sum(r["time"] for r in results) / len(results), 3)

    for r in results:
        if r["success"]:
            print(f"{r['id']:02d}: OK   {r.get('parcel_id', '')} ({r['time']}s)")
        else:
            print(f"{r['id']:02d}: FAIL {r.get('status_code')} {r.get('error', '')} ({r['time']}s)")

    print("\nSummary")
    print(f"Successes: {success_count}/{total_requests}")
    print(f"Total wall-clock time: {wall_clock}s")
```

```
    print(f"Average response time: {avg_response_time}s")


if __name__ == "__main__":
    main()
```

Appendix D - CloudFormation YAML Template

```yaml
AWSTemplateFormatVersion: "2010-09-09"
Description: SmartParcel NET214 Serverless Project

Parameters:
  StudentId:
    Type: String
    Default: khalid20200001899

  NotificationEmail:
    Type: String
    Default: 20200001899@students.cud.ac.ae

Resources:

  ParcelsTable:
    Type: AWS::DynamoDB::Table
    Properties:
      TableName: !Sub smartparcel-parcels-${StudentId}
      BillingMode: PAY_PER_REQUEST
      AttributeDefinitions:
        - AttributeName: parcel_id
          AttributeType: S
        - AttributeName: status
          AttributeType: S
      KeySchema:
        - AttributeName: parcel_id
          KeyType: HASH
      GlobalSecondaryIndexes:
        - IndexName: status-index
          KeySchema:
            - AttributeName: status
              KeyType: HASH
          Projection:
            ProjectionType: ALL

  PhotosBucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: !Sub smartparcel-photos-${StudentId}
      VersioningConfiguration:
        Status: Enabled
      BucketEncryption:
        ServerSideEncryptionConfiguration:
          - ServerSideEncryptionByDefault:
              SSEAlgorithm: AES256
      PublicAccessBlockConfiguration:
        BlockPublicAcls: true
        BlockPublicPolicy: true
        IgnorePublicAcls: true
        RestrictPublicBuckets: true
      LifecycleConfiguration:
        Rules:
          - Id: archive-old-photos
            Status: Enabled
            Transitions:
              - StorageClass: STANDARD_IA
                TransitionInDays: 30
              - StorageClass: GLACIER
                TransitionInDays: 90
            ExpirationInDays: 365

  EventsDLQ:
    Type: AWS::SQS::Queue
    Properties:
      QueueName: !Sub smartparcel-events-dlq-${StudentId}

  EventsQueue:
    Type: AWS::SQS::Queue
    Properties:
      QueueName: !Sub smartparcel-events-${StudentId}
      RedrivePolicy:
        deadLetterTargetArn: !GetAtt EventsDLQ.Arn
        maxReceiveCount: 3

  AlertsTopic:
    Type: AWS::SNS::Topic
    Properties:
      TopicName: !Sub smartparcel-alerts-${StudentId}

  AlertsSubscription:
    Type: AWS::SNS::Subscription
    Properties:
      TopicArn: !Ref AlertsTopic
      Protocol: email
      Endpoint: !Ref NotificationEmail

  ApiLambdaRole:
    Type: AWS::IAM::Role
    Properties:
      RoleName: !Sub smartparcel-api-role-${StudentId}
      AssumeRolePolicyDocument:
        Version: "2012-10-17"
        Statement:
          - Effect: Allow
            Principal:
              Service: lambda.amazonaws.com
            Action: sts:AssumeRole
```

```
    Policies:
      - PolicyName: smartparcel-api-policy
        PolicyDocument:
          Version: "2012-10-17"
          Statement:
            - Effect: Allow
              Action:
                - dynamodb:PutItem
                - dynamodb:GetItem
                - dynamodb:UpdateItem
                - dynamodb:DeleteItem
                - dynamodb:Scan
                - dynamodb:Query
              Resource:
                - !GetAtt ParcelsTable.Arn
                - !Sub "${ParcelsTable.Arn}/index/status-index"
            - Effect: Allow
              Action:
                - sqs:SendMessage
              Resource: !GetAtt EventsQueue.Arn
            - Effect: Allow
              Action:
                - s3:PutObject
                - s3:GetObject
                - s3:DeleteObject
              Resource: !Sub "${PhotosBucket.Arn}/*"
            - Effect: Allow
              Action:
                - logs:CreateLogGroup
                - logs:CreateLogStream
                - logs:PutLogEvents
              Resource: !Sub "arn:aws:logs:${AWS::Region}:${AWS::AccountId}:log-group:/aws/lambda/smartparcel-api-${StudentId}:*"

NotifierLambdaRole:
  Type: AWS::IAM::Role
  Properties:
    RoleName: !Sub smartparcel-notifier-role-${StudentId}
    AssumeRolePolicyDocument:
      Version: "2012-10-17"
      Statement:
        - Effect: Allow
          Principal:
            Service: lambda.amazonaws.com
          Action: sts:AssumeRole
    Policies:
      - PolicyName: smartparcel-notifier-policy
        PolicyDocument:
          Version: "2012-10-17"
          Statement:
            - Effect: Allow
              Action:
                - sqs:ReceiveMessage
                - sqs:DeleteMessage
                - sqs:GetQueueAttributes
              Resource: !GetAtt EventsQueue.Arn
            - Effect: Allow
              Action:
                - sns:Publish
              Resource: !Ref AlertsTopic
            - Effect: Allow
              Action:
                - logs:CreateLogGroup
                - logs:CreateLogStream
                - logs:PutLogEvents
              Resource: !Sub "arn:aws:logs:${AWS::Region}:${AWS::AccountId}:log-group:/aws/lambda/smartparcel-notifier-${StudentId}:*"

ApiLambda:
  Type: AWS::Lambda::Function
  Properties:
    FunctionName: !Sub smartparcel-api-${StudentId}
    Runtime: python3.12
    Handler: index.lambda_handler
    Role: !GetAtt ApiLambdaRole.Arn
    Timeout: 20
    Environment:
      Variables:
        TABLE_NAME: !Ref ParcelsTable
        QUEUE_URL: !Ref EventsQueue
    Code:
      ZipFile: |
        import json, os, uuid
        import boto3
        from boto3.dynamodb.conditions import Key
        from datetime import datetime, timezone

        dynamodb = boto3.resource("dynamodb")
        sqs = boto3.client("sqs")

        TABLE_NAME = os.environ["TABLE_NAME"]
        QUEUE_URL = os.environ["QUEUE_URL"]
        table = dynamodb.Table(TABLE_NAME)

        VALID_STATUSES = ["registered", "picked_up", "in_transit", "delivered"]

        def response(code, body):
            return {
                "statusCode": code,
                "headers": {"Content-Type": "application/json"},
                "body": json.dumps(body)
            }
```

```
def parse_body(event):
    body = event.get("body") or "{}"
    if len(body.encode("utf-8")) > 10240:
        raise ValueError("Payload too large. Maximum allowed size is 10 KB.")
    try:
        return json.loads(body)
    except:
        raise ValueError("Invalid JSON body")

def role(event):
    headers = event.get("headers") or {}
    return headers.get("X-User-Role") or headers.get("x-user-role") or ""

def require_role(event, allowed):
    user_role = role(event)
    if user_role not in allowed:
        return False
    return True

def require_fields(body, fields):
    for field in fields:
        if field not in body or body[field] in ["", None]:
            raise ValueError(f"Missing field: {field}")

def lambda_handler(event, context):
    try:
        method = event.get("httpMethod")
        path = event.get("resource")
        params = event.get("pathParameters") or {}

        if method == "POST" and path == "/parcels":
            if not require_role(event, ["driver", "admin"]):
                return response(403, {"error": "Forbidden"})
            return create_parcel(event)

        if method == "GET" and path == "/parcels/{id}":
            if not require_role(event, ["driver", "customer", "admin"]):
                return response(403, {"error": "Forbidden"})
            return get_parcel(params.get("id"))

        if method == "GET" and path == "/parcels":
            if not require_role(event, ["admin"]):
                return response(403, {"error": "Forbidden"})
            return list_parcels(event)

        if method == "PUT" and path == "/parcels/{id}/status":
            if not require_role(event, ["driver"]):
                return response(403, {"error": "Forbidden"})
            return update_status(event, params.get("id"))

        if method == "DELETE" and path == "/parcels/{id}":
            if not require_role(event, ["admin"]):
                return response(403, {"error": "Forbidden"})
            return delete_parcel(params.get("id"))

        return response(404, {"error": "Not found"})

    except ValueError as e:
        return response(400, {"error": str(e)})
    except Exception as e:
        print("ERROR:", str(e))
        return response(500, {"error": "Internal server error"})

def create_parcel(event):
    body = parse_body(event)
    require_fields(body, ["sender", "receiver", "address", "email"])

    parcel_id = "PKG-" + datetime.now().strftime("%Y") + "-" + uuid.uuid4().hex[:8].upper()
    now = datetime.now(timezone.utc).isoformat()

    item = {
        "parcel_id": parcel_id,
        "sender": body["sender"],
        "receiver": body["receiver"],
        "address": body["address"],
        "customer_email": body["email"],
        "status": "registered",
        "created_at": now,
        "updated_at": now
    }

    table.put_item(Item=item)
    return response(201, item)

def get_parcel(parcel_id):
    result = table.get_item(Key={"parcel_id": parcel_id})
    item = result.get("Item")
    if not item:
        return response(404, {"error": "Parcel not found"})
    return response(200, item)

def list_parcels(event):
    query = event.get("queryStringParameters") or {}
    status = query.get("status")

    if status:
        if status not in VALID_STATUSES:
            return response(400, {"error": "Invalid status"})
        result = table.query(
            IndexName="status-index",
            KeyConditionExpression=Key("status").eq(status)
```

```
            )
        else:
            result = table.scan()

        return response(200, {"items": result.get("Items", [])})

    def update_status(event, parcel_id):
        body = parse_body(event)
        # Rubric/lab uses "new_status". "status" is also accepted to avoid test mistakes.
        new_status = body.get("new_status") or body.get("status")
        if not new_status:
            raise ValueError("Missing field: new_status")
        if new_status not in VALID_STATUSES:
            return response(400, {"error": "Invalid status"})

        result = table.get_item(Key={"parcel_id": parcel_id})
        item = result.get("Item")
        if not item:
            return response(404, {"error": "Parcel not found"})

        old_status = item["status"]
        if VALID_STATUSES.index(new_status) <= VALID_STATUSES.index(old_status):
            return response(409, {"error": "Status can only move forward"})

        now = datetime.now(timezone.utc).isoformat()

        table.update_item(
            Key={"parcel_id": parcel_id},
            UpdateExpression="SET #s = :s, updated_at = :u",
            ExpressionAttributeNames={"#s": "status"},
            ExpressionAttributeValues={":s": new_status, ":u": now}
        )

        sqs.send_message(
            QueueUrl=QUEUE_URL,
            MessageBody=json.dumps({
                "parcel_id": parcel_id,
                "new_status": new_status,
                "customer_email": item["customer_email"],
                "timestamp": now
            })
        )

        item["status"] = new_status
        item["updated_at"] = now
        return response(200, item)

    def delete_parcel(parcel_id):
        result = table.get_item(Key={"parcel_id": parcel_id})
        item = result.get("Item")
        if not item:
            return response(404, {"error": "Parcel not found"})

        if item["status"] != "registered":
            return response(409, {"error": "Only registered parcels can be cancelled"})

        table.delete_item(Key={"parcel_id": parcel_id})
        return response(200, {"message": "Parcel cancelled", "parcel_id": parcel_id})

NotifierLambda:
  Type: AWS::Lambda::Function
  Properties:
    FunctionName: !Sub smartparcel-notifier-${StudentId}
    Runtime: python3.12
    Handler: index.lambda_handler
    Role: !GetAtt NotifierLambdaRole.Arn
    Timeout: 20
    Environment:
      Variables:
        TOPIC_ARN: !Ref AlertsTopic
    Code:
      ZipFile: |
        import json, os
        import boto3

        sns = boto3.client("sns")
        TOPIC_ARN = os.environ["TOPIC_ARN"]

        def lambda_handler(event, context):
            for record in event.get("Records", []):
                try:
                    body = json.loads(record["body"])
                    parcel_id = body["parcel_id"]
                    new_status = body["new_status"]
                    customer_email = body.get("customer_email", "customer")

                    message = f"SmartParcel update: Parcel {parcel_id} is now {new_status}. Customer: {customer_email}"

                    sns.publish(
                        TopicArn=TOPIC_ARN,
                        Subject="SmartParcel Status Update",
                        Message=message
                    )

                except Exception as e:
                    print("Malformed message skipped:", str(e))
                    continue

            return {"statusCode": 200}

SqsTrigger:
```

```
  Type: AWS::Lambda::EventSourceMapping
  Properties:
    EventSourceArn: !GetAtt EventsQueue.Arn
    FunctionName: !Ref NotifierLambda
    BatchSize: 10
    Enabled: true

SmartParcelApi:
  Type: AWS::ApiGateway::RestApi
  Properties:
    Name: !Sub SmartParcel-API-${StudentId}
    Body:
      swagger: "2.0"
      info:
        title: !Sub SmartParcel-API-${StudentId}
        version: "1.0"
      schemes:
        - https
      paths:
        /parcels:
          post:
            security:
              - api_key: []
            x-amazon-apigateway-integration:
              uri: !Sub arn:aws:apigateway:${AWS::Region}:lambda:path/2015-03-31/functions/${ApiLambda.Arn}/invocations
              httpMethod: POST
              type: aws_proxy
          get:
            security:
              - api_key: []
            x-amazon-apigateway-integration:
              uri: !Sub arn:aws:apigateway:${AWS::Region}:lambda:path/2015-03-31/functions/${ApiLambda.Arn}/invocations
              httpMethod: POST
              type: aws_proxy
        /parcels/{id}:
          get:
            parameters:
              - name: id
                in: path
                required: true
                type: string
            x-amazon-apigateway-integration:
              uri: !Sub arn:aws:apigateway:${AWS::Region}:lambda:path/2015-03-31/functions/${ApiLambda.Arn}/invocations
              httpMethod: POST
              type: aws_proxy
          delete:
            security:
              - api_key: []
            parameters:
              - name: id
                in: path
                required: true
                type: string
            x-amazon-apigateway-integration:
              uri: !Sub arn:aws:apigateway:${AWS::Region}:lambda:path/2015-03-31/functions/${ApiLambda.Arn}/invocations
              httpMethod: POST
              type: aws_proxy
        /parcels/{id}/status:
          put:
            security:
              - api_key: []
            parameters:
              - name: id
                in: path
                required: true
                type: string
            x-amazon-apigateway-integration:
              uri: !Sub arn:aws:apigateway:${AWS::Region}:lambda:path/2015-03-31/functions/${ApiLambda.Arn}/invocations
              httpMethod: POST
              type: aws_proxy
      securityDefinitions:
        api_key:
          type: apiKey
          name: x-api-key
          in: header

ApiDeployment:
  Type: AWS::ApiGateway::Deployment
  DependsOn:
    - SmartParcelApi
  Properties:
    RestApiId: !Ref SmartParcelApi
    StageName: prod

ApiKey:
  Type: AWS::ApiGateway::ApiKey
  Properties:
    Name: !Sub smartparcel-key-${StudentId}
    Enabled: true

UsagePlan:
  Type: AWS::ApiGateway::UsagePlan
  DependsOn: ApiDeployment
  Properties:
    UsagePlanName: !Sub smartparcel-usage-plan-${StudentId}
    ApiStages:
      - ApiId: !Ref SmartParcelApi
        Stage: prod

UsagePlanKey:
  Type: AWS::ApiGateway::UsagePlanKey
```

```yaml
  Properties:
    KeyId: !Ref ApiKey
    KeyType: API_KEY
    UsagePlanId: !Ref UsagePlan

LambdaApiPermission:
  Type: AWS::Lambda::Permission
  Properties:
    FunctionName: !Ref ApiLambda
    Action: lambda:InvokeFunction
    Principal: apigateway.amazonaws.com
    SourceArn: !Sub arn:aws:execute-api:${AWS::Region}:${AWS::AccountId}:${SmartParcelApi}/*/*/*

ApiErrorAlarm:
  Type: AWS::CloudWatch::Alarm
  Properties:
    AlarmName: !Sub smartparcel-errors-${StudentId}
    MetricName: Errors
    Namespace: AWS/Lambda
    Statistic: Sum
    Period: 300
    EvaluationPeriods: 1
    Threshold: 1
    ComparisonOperator: GreaterThanOrEqualToThreshold
    Dimensions:
      - Name: FunctionName
        Value: !Ref ApiLambda
    AlarmActions:
      - !Ref AlertsTopic

SmartParcelDashboard:
  Type: AWS::CloudWatch::Dashboard
  Properties:
    DashboardName: !Sub smartparcel-dashboard-${StudentId}
    DashboardBody: !Sub |
      {
        "widgets": [
          {
            "type": "metric",
            "x": 0,
            "y": 0,
            "width": 8,
            "height": 6,
            "properties": {
              "region": "${AWS::Region}",
              "title": "API Lambda Invocations",
              "metrics": [["AWS/Lambda", "Invocations", "FunctionName", "smartparcel-api-${StudentId}"]],
              "stat": "Sum",
              "period": 300
            }
          },
          {
            "type": "metric",
            "x": 8,
            "y": 0,
            "width": 8,
            "height": 6,
            "properties": {
              "region": "${AWS::Region}",
              "title": "API Lambda Errors",
              "metrics": [["AWS/Lambda", "Errors", "FunctionName", "smartparcel-api-${StudentId}"]],
              "stat": "Sum",
              "period": 300
            }
          },
          {
            "type": "metric",
            "x": 16,
            "y": 0,
            "width": 8,
            "height": 6,
            "properties": {
              "region": "${AWS::Region}",
              "title": "SQS Sent vs Received",
              "metrics": [["AWS/SQS", "NumberOfMessagesSent", "QueueName", "smartparcel-events-${StudentId}"], [".", "NumberOfMessagesReceived", ".", "smartparcel-events-${StudentId}"]],
              "stat": "Sum",
              "period": 300
            }
          }
        ]
      }

SmartParcelVpc:
  Type: AWS::EC2::VPC
  Properties:
    CidrBlock: 10.0.0.0/16
    Tags:
      - Key: Name
        Value: !Sub smartparcel-vpc-${StudentId}

PublicSubnet:
  Type: AWS::EC2::Subnet
  Properties:
    VpcId: !Ref SmartParcelVpc
    CidrBlock: 10.0.1.0/24
    AvailabilityZone: !Sub ${AWS::Region}a
    Tags:
      - Key: Name
        Value: !Sub smartparcel-public-subnet-${StudentId}

PrivateSubnet:
```

```
  Type: AWS::EC2::Subnet
  Properties:
    VpcId: !Ref SmartParcelVpc
    CidrBlock: 10.0.2.0/24
    AvailabilityZone: !Sub ${AWS::Region}b
    Tags:
      - Key: Name
        Value: !Sub smartparcel-private-subnet-${StudentId}

PrivateRouteTable:
  Type: AWS::EC2::RouteTable
  Properties:
    VpcId: !Ref SmartParcelVpc
    Tags:
      - Key: Name
        Value: !Sub smartparcel-private-rt-${StudentId}

PrivateSubnetAssociation:
  Type: AWS::EC2::SubnetRouteTableAssociation
  Properties:
    SubnetId: !Ref PrivateSubnet
    RouteTableId: !Ref PrivateRouteTable

InternetGateway:
  Type: AWS::EC2::InternetGateway
  Properties:
    Tags:
      - Key: Name
        Value: !Sub smartparcel-igw-${StudentId}

AttachGateway:
  Type: AWS::EC2::VPCGatewayAttachment
  Properties:
    VpcId: !Ref SmartParcelVpc
    InternetGatewayId: !Ref InternetGateway

PublicRouteTable:
  Type: AWS::EC2::RouteTable
  Properties:
    VpcId: !Ref SmartParcelVpc
    Tags:
      - Key: Name
        Value: !Sub smartparcel-public-rt-${StudentId}

PublicRoute:
  Type: AWS::EC2::Route
  DependsOn: AttachGateway
  Properties:
    RouteTableId: !Ref PublicRouteTable
    DestinationCidrBlock: 0.0.0.0/0
    GatewayId: !Ref InternetGateway

PublicSubnetAssociation:
  Type: AWS::EC2::SubnetRouteTableAssociation
  Properties:
    SubnetId: !Ref PublicSubnet
    RouteTableId: !Ref PublicRouteTable

PublicNetworkAcl:
  Type: AWS::EC2::NetworkAcl
  Properties:
    VpcId: !Ref SmartParcelVpc
    Tags:
      - Key: Name
        Value: !Sub smartparcel-public-nacl-${StudentId}

NaclInboundHTTP:
  Type: AWS::EC2::NetworkAclEntry
  Properties:
    NetworkAclId: !Ref PublicNetworkAcl
    RuleNumber: 100
    Protocol: 6
    RuleAction: allow
    Egress: false
    CidrBlock: 0.0.0.0/0
    PortRange:
      From: 80
      To: 80

NaclInboundHTTPS:
  Type: AWS::EC2::NetworkAclEntry
  Properties:
    NetworkAclId: !Ref PublicNetworkAcl
    RuleNumber: 110
    Protocol: 6
    RuleAction: allow
    Egress: false
    CidrBlock: 0.0.0.0/0
    PortRange:
      From: 443
      To: 443

NaclOutboundAll:
  Type: AWS::EC2::NetworkAclEntry
  Properties:
    NetworkAclId: !Ref PublicNetworkAcl
    RuleNumber: 100
    Protocol: -1
    RuleAction: allow
    Egress: true
    CidrBlock: 0.0.0.0/0
```

```
  PublicSubnetNaclAssociation:
    Type: AWS::EC2::SubnetNetworkAclAssociation
    Properties:
      SubnetId: !Ref PublicSubnet
      NetworkAclId: !Ref PublicNetworkAcl

  DynamoDbVpcEndpoint:
    Type: AWS::EC2::VPCEndpoint
    Properties:
      VpcId: !Ref SmartParcelVpc
      ServiceName: !Sub com.amazonaws.${AWS::Region}.dynamodb
      VpcEndpointType: Gateway
      RouteTableIds:
        - !Ref PublicRouteTable
        - !Ref PrivateRouteTable

Outputs:
  ApiInvokeUrl:
    Description: API Gateway invoke URL
    Value: !Sub https://${SmartParcelApi}.execute-api.${AWS::Region}.amazonaws.com/prod

  ApiKeyId:
    Description: API key ID
    Value: !Ref ApiKey

  SnsTopicArn:
    Description: SNS topic ARN
    Value: !Ref AlertsTopic
```